คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

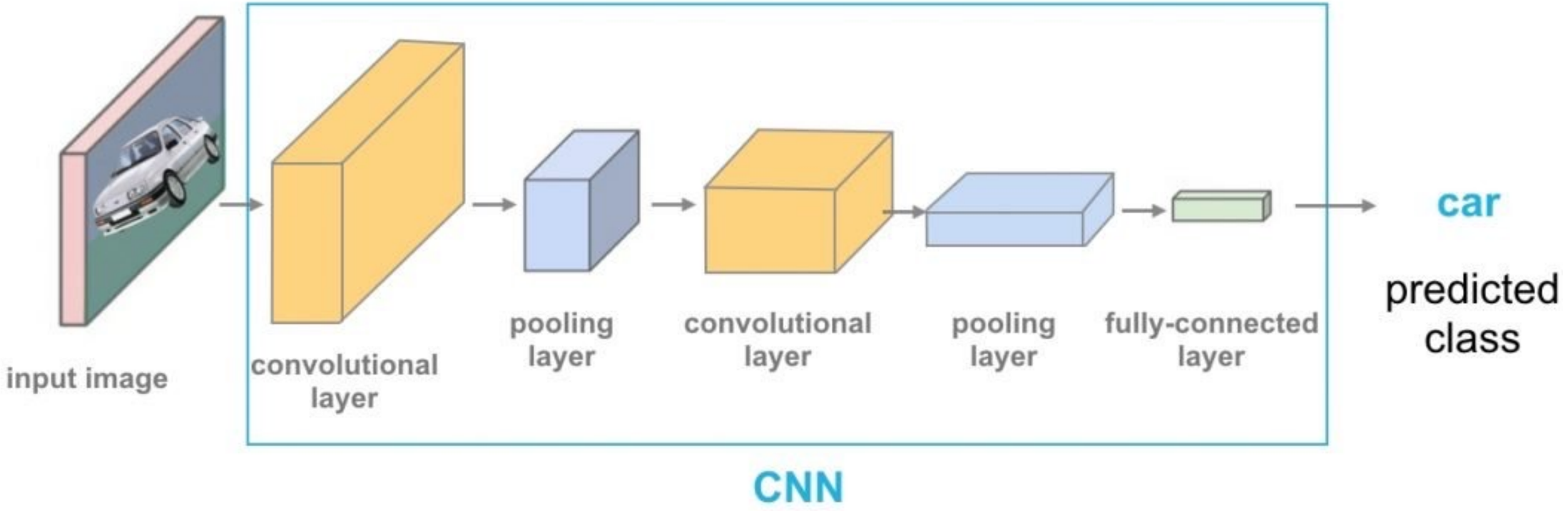
Chapter 14

Understanding Transformers for Computer Vision

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

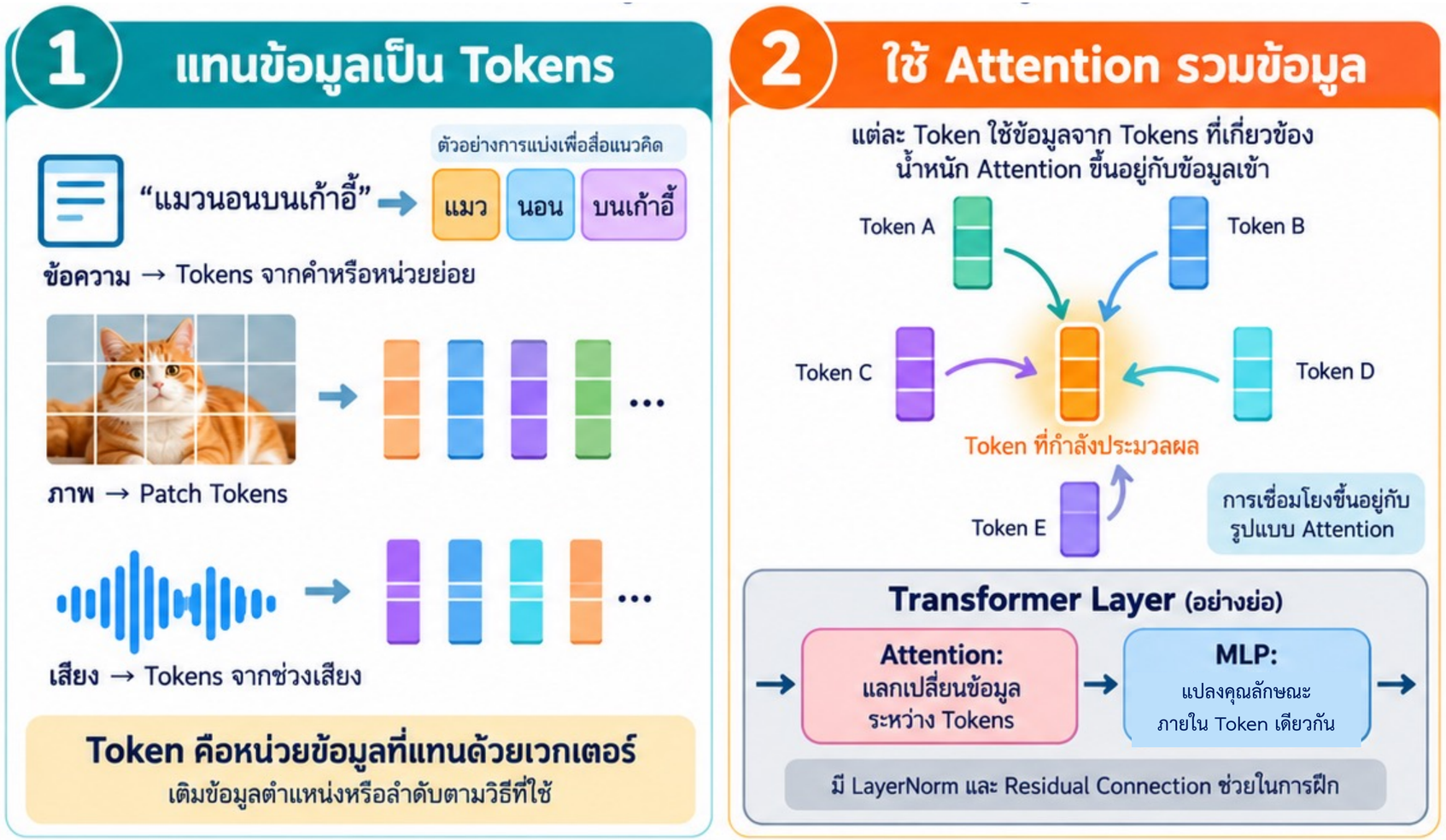
คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง


input image
convolutional layer
pooling layer
convolutional layer
pooling layer
fully-connected layer
car
predicted class
CNN

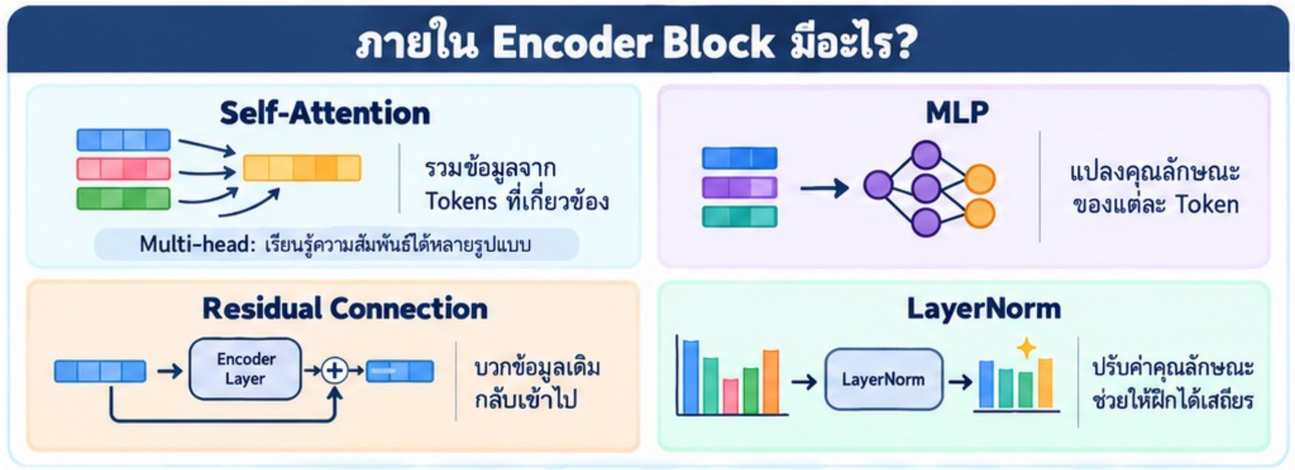
สถาปัตยกรรม Transformer คืออะไร

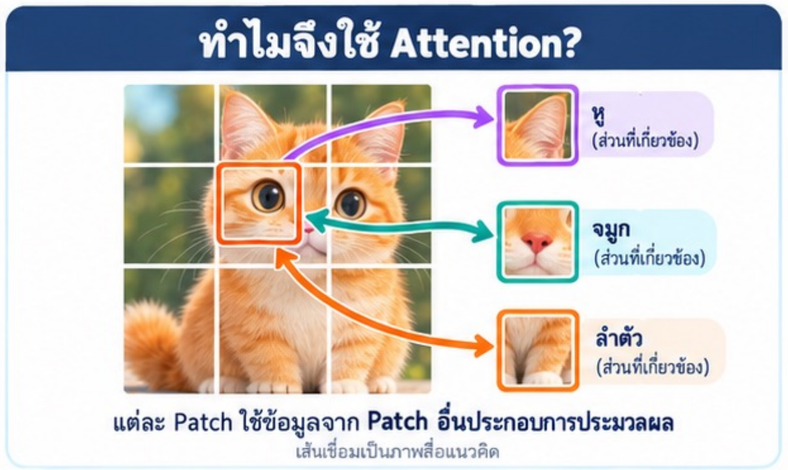
Transformer เป็นสถาปัตยกรรมโครงข่ายประสาทเทียมที่ใช้ Attention เรียนรู้ความสัมพันธ์และแลกเปลี่ยนข้อมูลระหว่าง Tokens (รวมข้อมูลระหว่าง Tokens) เพื่อสร้างคุณลักษณะที่เข้าใจถึงบริบท โดยสามารถประยุกต์ใช้กับข้อความ ภาพ และข้อมูลชนิดอื่นได้



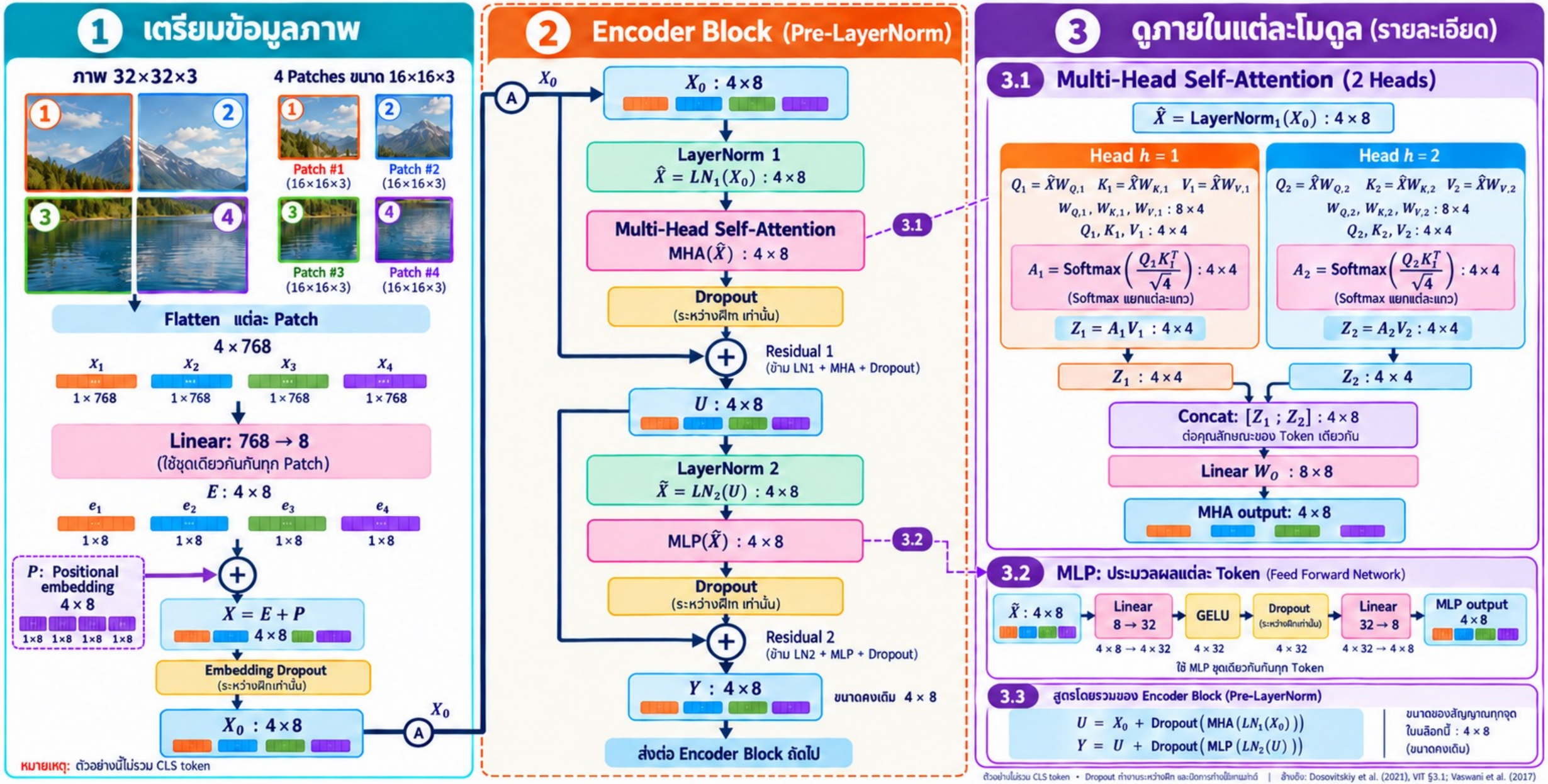
Transformer กับ Vision Transformer ต่างกันอย่างไร







Vision Transformer (ViT) คือ สถาปัตยกรรม Transformer ที่นำแนวคิด Attention มาใช้กับข้อมูลภาพ โดยแบ่งภาพเป็น Patches แปลงเป็น Tokens เพิ่มข้อมูลตำแหน่ง แล้วส่งผ่าน Transformer Encoder ก่อนนำผลไปใช้กับงาน ทั้งนี้ Transformer ต้นฉบับประกอบด้วย Encoder และ Decoder แต่โมเดลที่พัฒนาต่อมาอาจใช้เฉพาะ Encoder เช่น ViT หรือเฉพาะ Decoder ก็ได้


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกันทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4×8
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4×8
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3.1
3.2
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4 × 8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8 × 4
Q1, K1, V1 : 4 × 4
A1 = Softmax(Q1K1T / √4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4 × 4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8 × 4
Q2, K2, V2 : 4 × 4
A2 = Softmax(Q2K2T / √4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4 × 4
Z1 : 4×4
Z2 : 4 × 4
Concat: [Z1 ; Z2] : 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4 × 8
Linear 8 → 32
4 × 8 → 4 × 32
GELU
4 × 32
Dropout (ระหว่างฝึกเท่านั้น)
4 × 32
Linear 32 → 8
4 × 32 → 4 × 8
MLP output 4 × 8
ใช้ MLP ชุดเดียวกันกันทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อทำนาย | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

1 เตรียมข้อมูลภาพ

ภาพ 32×32×3



4 Patches ขนาด 16×16×3



Patch #1 (16×16×3) | Patch #2 (16×16×3)

Patch #3 (16×16×3) | Patch #4 (16×16×3)

Flatten แต่ละ Patch

4×768

x_1 x_2 x_3 x_4

1×768 1×768 1×768 1×768

Linear: 768 → 8
(ใช้ชุดเดียวกันกันทุก Patch)

$E : 4 \times 8$

e_1 e_2 e_3 e_4

1×8 1×8 1×8 1×8

P: Positional embedding 4×8

1×8 1×8 1×8 1×8

$X = E + P$ 4×8

Embedding Dropout
(ระหว่างฝึกเท่านั้น)

$X_0 : 4 \times 8$ → A → X_0

หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token

2 Encoder Block (Pre-LayerNorm)

X_0 → A

$X_0 : 4 \times 8$

LayerNorm 1
$\hat{X} = LN_1(X_0) : 4 \times 8$

Multi-Head Self-Attention (3.1)
$MHA(\hat{X}) : 4 \times 8$

Dropout
(ระหว่างฝึก เท่านั้น)

Residual 1
(ข้าม LN1 + MHA + Dropout)

$U : 4 \times 8$

LayerNorm 2
$\tilde{X} = LN_2(U) : 4 \times 8$

MLP (3.2)
$MLP(\tilde{X}) : 4 \times 8$

Dropout
(ระหว่างฝึก เท่านั้น)

Residual 2
(ข้าม LN2 + MLP + Dropout)

$Y : 4 \times 8$ ขนาดคงเดิม 4×8

ส่งต่อ Encoder Block ถัดไป

3 ดูภายในแต่ละโมดูล (รายละเอียด)

3.1 Multi-Head Self-Attention (2 Heads)

$\hat{X} = \text{LayerNorm}_1(X_0) : 4 \times 8$

Head $h = 1$

$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$

$W_{Q,1}, W_{K,1}, W_{V,1} : 8 \times 4$

$Q_1, K_1, V_1 : 4 \times 4$

$A_1 = \text{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right) : 4 \times 4$

(Softmax แยกแต่ละแถว)

$Z_1 = A_1 V_1 : 4 \times 4$

Head $h = 2$

$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$

$W_{Q,2}, W_{K,2}, W_{V,2} : 8 \times 4$

$Q_2, K_2, V_2 : 4 \times 4$

$A_2 = \text{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right) : 4 \times 4$

(Softmax แยกแต่ละแถว)

$Z_2 = A_2 V_2 : 4 \times 4$

$Z_1 : 4 \times 4$ $Z_2 : 4 \times 4$

Concat: $[Z_1 ; Z_2] : 4 \times 8$
ต่อคุณลักษณะของ Token เดียวกัน

Linear $W_O : 8 \times 8$

MHA output: 4×8

3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)

$\tilde{X} : 4 \times 8$ → Linear $8 \to 32$ ($4 \times 8 \to 4 \times 32$) → GELU ($4 \times 32$) → Dropout (ระหว่างฝึกเท่านั้น) (4×32) → Linear $32 \to 8$ ($4 \times 32 \to 4 \times 8$) → MLP output 4×8

ใช้ MLP ชุดเดียวกันกันทุก Token

3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)

$U = X_0 + \text{Dropout}\left(MHA\left(LN_1(X_0)\right)\right)$

$Y = U + \text{Dropout}\left(MLP\left(LN_2(U)\right)\right)$

ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4×8 (ขนาดคงเดิม)

ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานอัตโนมัติตอนใช้ | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

1 เตรียมข้อมูลภาพ

ภาพ 32×32×3



4 Patches ขนาด 16×16×3



Patch #1 (16×16×3)

Patch #2 (16×16×3)



Patch #3 (16×16×3)



Patch #4 (16×16×3)

Flatten แต่ละ Patch

4×768

x_1 x_2 x_3 x_4

1×768 1×768 1×768 1×768

Linear: $768 \rightarrow 8$

(ใช้ชุดเดียวกันกับทุก Patch)

$E : 4 \times 8$

e_1 e_2 e_3 e_4

1×8 1×8 1×8 1×8

P: Positional embedding 4×8

1×8 1×8 1×8 1×8

$X = E + P$ 4×8

Embedding Dropout

(ระหว่างฝึกเท่านั้น)

$X_0 : 4 \times 8$

X_0

หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token

2 Encoder Block (Pre-LayerNorm)

A X_0

$X_0 : 4 \times 8$

LayerNorm 1

$\hat{X} = LN_1(X_0) : 4 \times 8$

Multi-Head Self-Attention

$\mathrm{MHA}(\hat{X}) : 4 \times 8$ → 3.1

Dropout

(ระหว่างฝึก เท่านั้น)

Residual 1

(ข้าม LN1 + MHA + Dropout)

$U : 4 \times 8$

LayerNorm 2

$\tilde{X} = LN_2(U) : 4 \times 8$

$\mathrm{MLP}(\tilde{X}) : 4 \times 8$ → 3.2

Dropout

(ระหว่างฝึก เท่านั้น)

Residual 2

(ข้าม LN2 + MLP + Dropout)

$Y : 4 \times 8$

ขนาดคงเดิม 4×8

ส่งต่อ Encoder Block ถัดไป

3 ดูภายในแต่ละโมดูล (รายละเอียด)

3.1 Multi-Head Self-Attention (2 Heads)

$\hat{X} = \mathrm{LayerNorm}_1(X_0) : 4 \times 8$

Head $h = 1$

$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$

$W_{Q,1}, W_{K,1}, W_{V,1} : 8 \times 4$

$Q_1, K_1, V_1 : 4 \times 4$

$A_1 = \mathrm{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right) : 4 \times 4$

(Softmax แยกแต่ละแถว)

$Z_1 = A_1 V_1 : 4 \times 4$

Head $h = 2$

$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$

$W_{Q,2}, W_{K,2}, W_{V,2} : 8 \times 4$

$Q_2, K_2, V_2 : 4 \times 4$

$A_2 = \mathrm{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right) : 4 \times 4$

(Softmax แยกแต่ละแถว)

$Z_2 = A_2 V_2 : 4 \times 4$

$Z_1 : 4 \times 4$ $Z_2 : 4 \times 4$

Concat: $[Z_1 ; Z_2] : 4 \times 8$

ต่อคุณลักษณะของ Token เดียวกัน

Linear $W_O : 8 \times 8$

MHA output: 4×8

3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)

$\tilde{X} : 4 \times 8$ → Linear $8 \rightarrow 32$ ($4 \times 8 \rightarrow 4 \times 32$) → GELU ($4 \times 32$) → Dropout (ระหว่างฝึกเท่านั้น) (4×32) → Linear $32 \rightarrow 8$ ($4 \times 32 \rightarrow 4 \times 8$) → MLP output 4×8

ใช้ MLP ชุดเดียวกันกับทุก Token

3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)

$$U = X_0 + \mathrm{Dropout}\left(\mathrm{MHA}\left(LN_1(X_0)\right)\right)$$

$$Y = U + \mathrm{Dropout}\left(\mathrm{MLP}\left(LN_2(U)\right)\right)$$

ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4×8 (ขนาดคงเดิม)

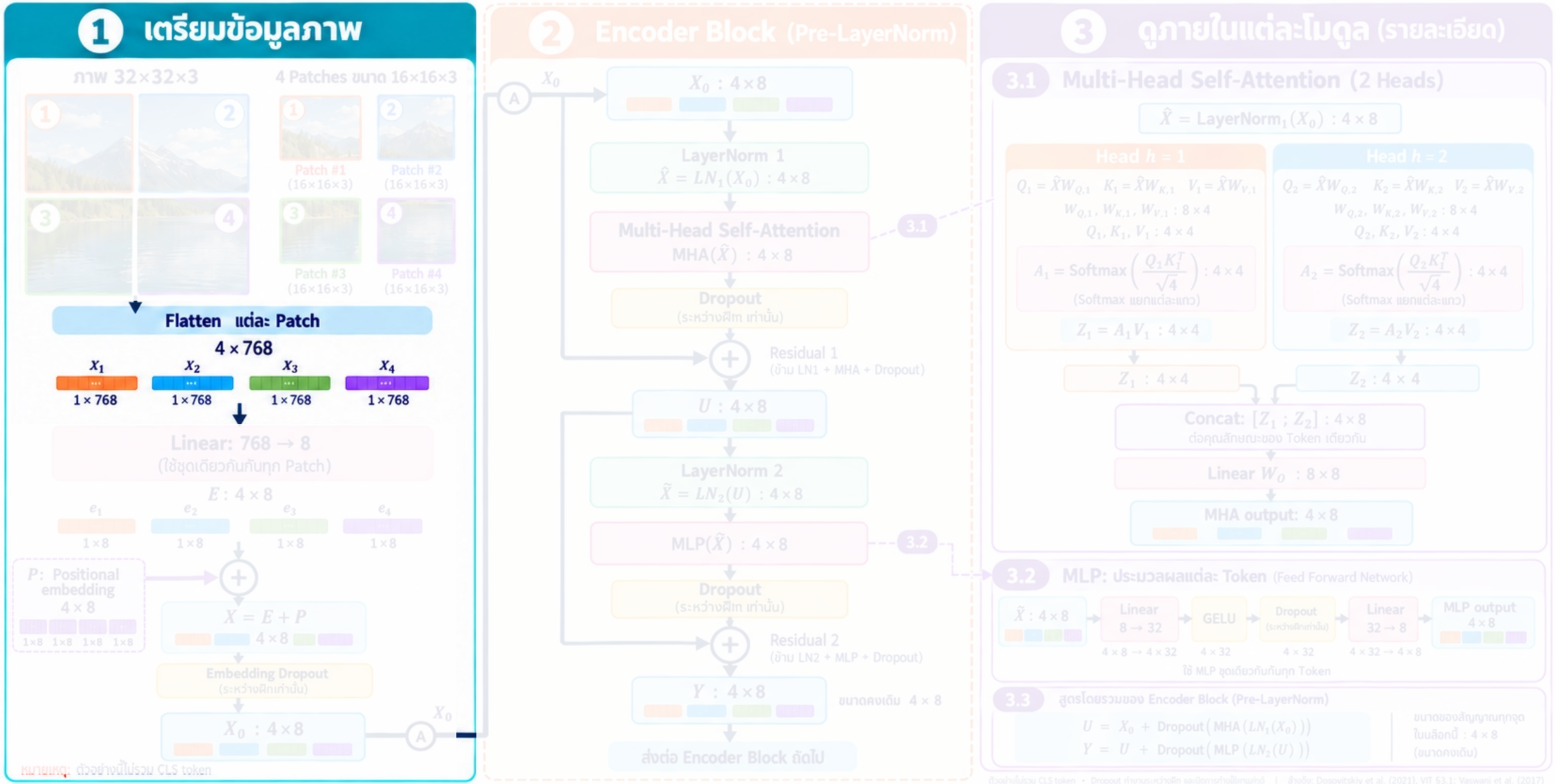
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานอัตโนมัติตอนใช้งาน | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

ขั้นตอนการแปลง Image เป็น Patch

Image to Patches คือ การดึงบริเวณย่อยจากภาพ โดยเชื่อมกับแนวคิดการแบ่งภาพเป็นบล็อกและการเลื่อนหน้าต่าง (sliding window) เพื่อใช้ประมวลผลบริเวณย่อย ViT โดยที่ ใน ViT ใช้ non-overlapping



Image generated by OpenAI (2026).


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
x_1 x_2 x_3 x_4
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
e_1 e_2 e_3 e_4
1×8
P: Positional embedding
4 × 8
1×8 1×8 1×8 1×8
$X = E + P$
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X_0 : 4×8
X_0
A
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X_0
A
X_0 : 4×8
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4×8
Multi-Head Self-Attention
$MHA(\hat{X})$: 4×8
3.1
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4×8
$MLP(\tilde{X})$: 4×8
3.2
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
$\hat{X} = \text{LayerNorm}_1(X_0)$: 4×8
Head $h = 1$
$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$
$W_{Q,1}, W_{K,1}, W_{V,1}$: 8×4
Q_1, K_1, V_1 : 4×4
$A_1 = \text{Softmax}\left(\frac{Q_1K_1^T}{\sqrt{4}}\right)$: 4×4
(Softmax แยกแต่ละแถว)
$Z_1 = A_1V_1$: 4×4
Head $h = 2$
$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$
$W_{Q,2}, W_{K,2}, W_{V,2}$: 8×4
Q_2, K_2, V_2 : 4×4
$A_2 = \text{Softmax}\left(\frac{Q_2K_2^T}{\sqrt{4}}\right)$: 4×4
(Softmax แยกแต่ละแถว)
$Z_2 = A_2V_2$: 4×4
Z_1 : 4×4
Z_2 : 4×4
Concat: $[Z_1 ; Z_2]$: 4×8
ต่อคุณลักษณะของ Token เดียวกัน
Linear W_O : 8×8
MHA output: 4×8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
$\tilde{X}$: 4×8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4×8
4×8 → 4×32
4×32
4×32
4×32 → 4×8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
$U = X_0 + \text{Dropout}(MHA(LN_1(X_0)))$
$Y = U + \text{Dropout}(MLP(LN_2(U)))$
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

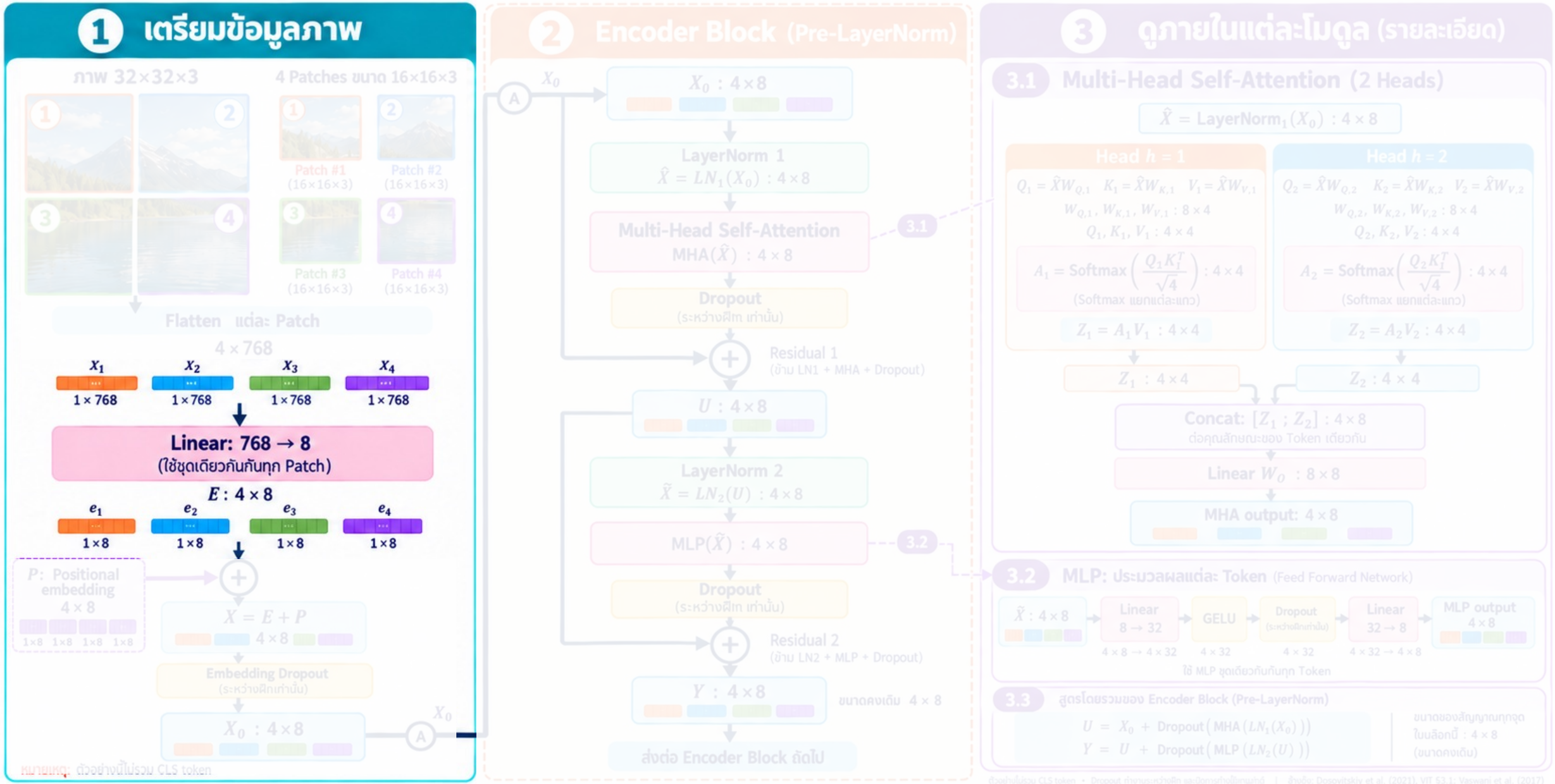

1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
x_1 x_2 x_3 x_4
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกันทุก Patch)
E : 4 × 8
e_1 e_2 e_3 e_4
1×8
P: Positional embedding
4 × 8
$X = E + P$
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X_0 : 4×8
X_0
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X_0
X_0 : 4×8
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4×8
Multi-Head Self-Attention
$MHA(\hat{X})$: 4 × 8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4×8
$MLP(\tilde{X})$: 4 × 8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
$\hat{X} = \text{LayerNorm}_1(X_0)$: 4 × 8
Head $h = 1$
$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$
$W_{Q,1}, W_{K,1}, W_{V,1}$: 8 × 4
Q_1, K_1, V_1 : 4 × 4
$A_1 = \text{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_1 = A_1 V_1$: 4 × 4
Head $h = 2$
$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$
$W_{Q,2}, W_{K,2}, W_{V,2}$: 8 × 4
Q_2, K_2, V_2 : 4 × 4
$A_2 = \text{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_2 = A_2 V_2$: 4 × 4
Z_1 : 4 × 4
Z_2 : 4 × 4
Concat: $[Z_1 ; Z_2]$: 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear W_O : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
$\tilde{X}$: 4 × 8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4 × 8
4 × 8 → 4 × 32
4 × 32
4 × 32
4 × 32 → 4 × 8
ใช้ MLP ชุดเดียวกันกันทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
$U = X_0 + \text{Dropout}(MHA(LN_1(X_0)))$
$Y = U + \text{Dropout}(MLP(LN_2(U)))$
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

Image Embedding คืออะไร

Image Embedding คือ การแทนภาพด้วยเวกเตอร์คุณลักษณะ ไม่ใช้เป็นการย่อขนาดไฟล์ภาพโดยแนวคิดเชื่อมกับการแทนข้อมูลและการเรียนรู้คุณลักษณะ (Representation Learning) ซึ่งมีมาก่อน Transformer โดยที่

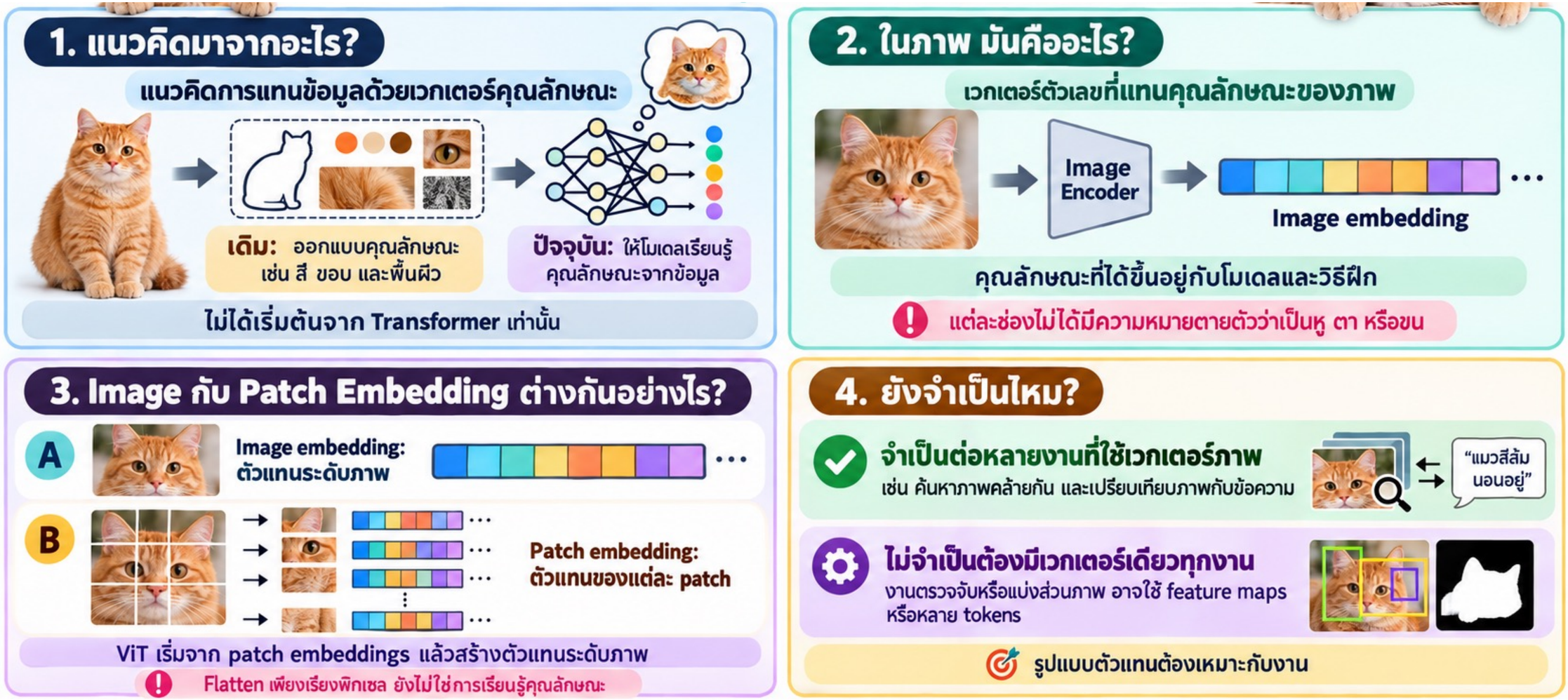


Image generated by OpenAI (2026).

Patch Embedding คืออะไร

Patch Embedding คือการแปลงพิกเซลของแต่ละ patch เป็นเวกเตอร์คุณลักษณะ เพื่อให้ Transformer ประมวลผลเป็นลำดับ tokens ได้ แนวคิดใน ViT เชื่อมมาจากการแทน tokens ในภาษาด้วยเวกเตอร์

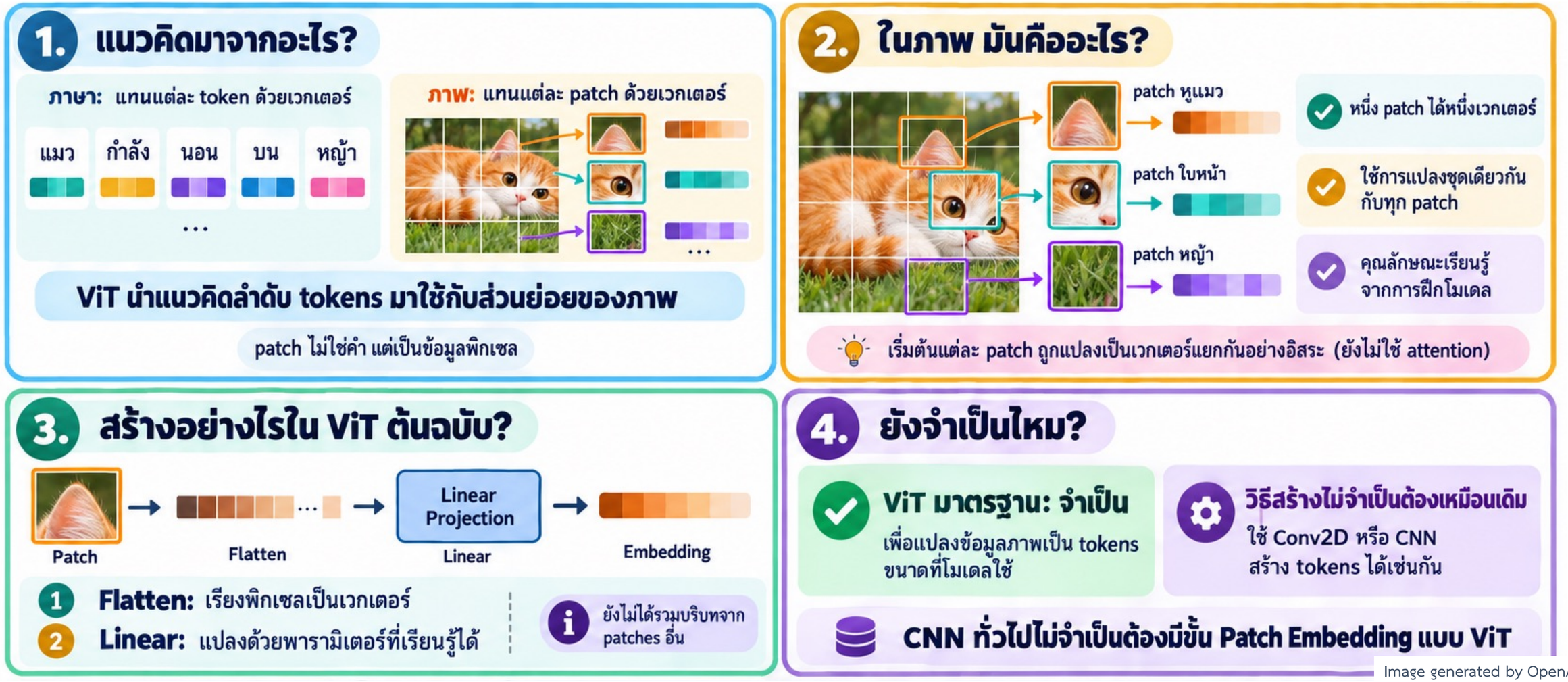
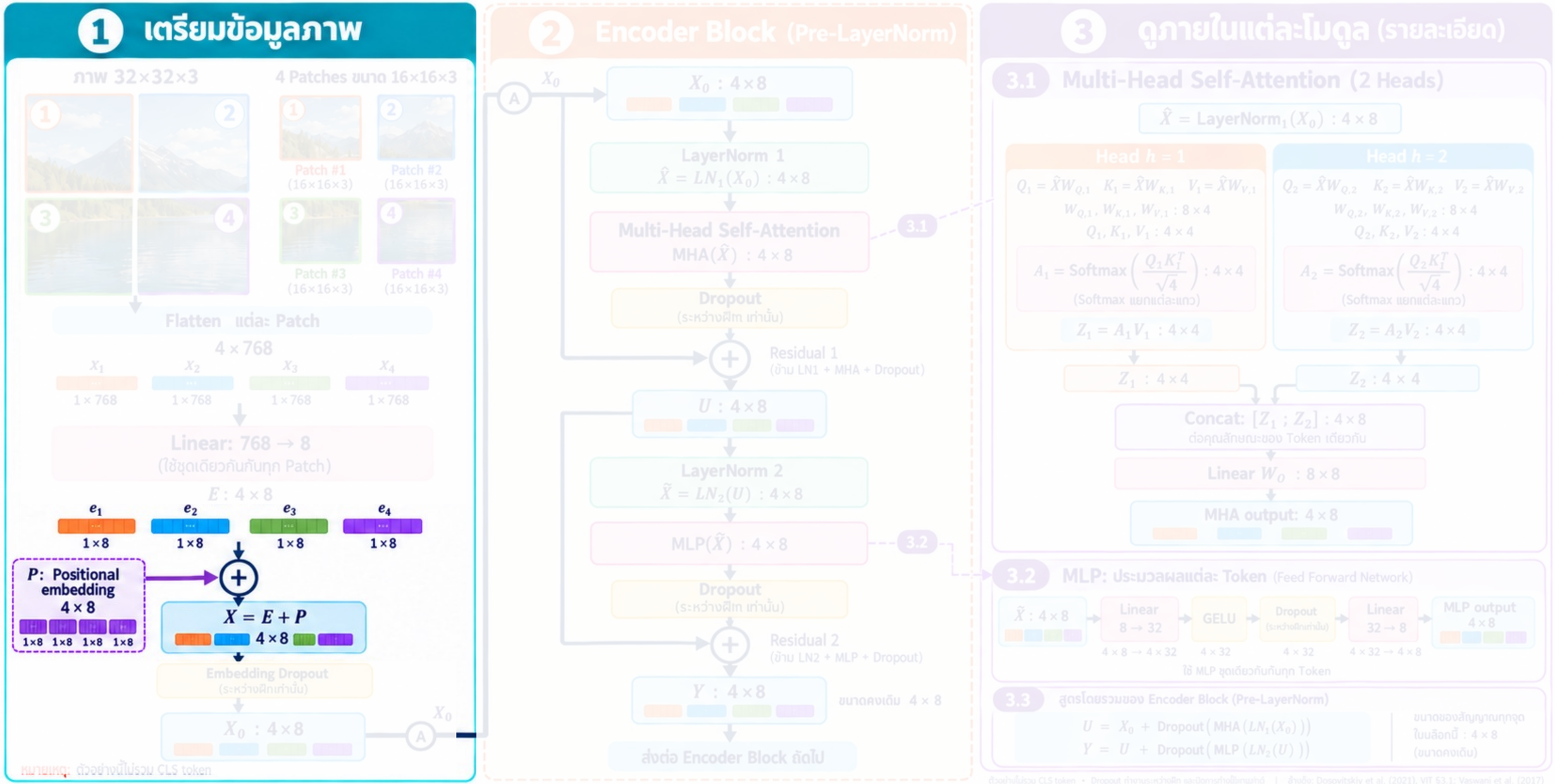


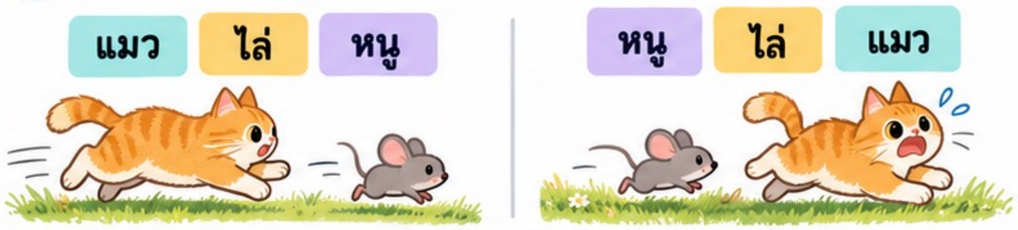
Image generated by OpenAI (2026).


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
x_1 x_2 x_3 x_4
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
e_1 e_2 e_3 e_4
1×8
P: Positional embedding
4 × 8
$X = E + P$
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X_0 : 4 × 8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X_0
X_0 : 4 × 8
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4 × 8
Multi-Head Self-Attention
MHA($\hat{X}$) : 4 × 8
3.1
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4 × 8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4 × 8
MLP($\tilde{X}$) : 4 × 8
3.2
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4 × 8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
$\hat{X} = \text{LayerNorm}_1(X_0)$: 4 × 8
Head $h = 1$
$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$
$W_{Q,1}, W_{K,1}, W_{V,1}$: 8 × 4
Q_1, K_1, V_1 : 4 × 4
$A_1 = \text{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_1 = A_1 V_1$: 4 × 4
Head $h = 2$
$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$
$W_{Q,2}, W_{K,2}, W_{V,2}$: 8 × 4
Q_2, K_2, V_2 : 4 × 4
$A_2 = \text{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_2 = A_2 V_2$: 4 × 4
Z_1 : 4 × 4
Z_2 : 4 × 4
Concat: $[Z_1 ; Z_2]$: 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear W_O : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
$\tilde{X}$: 4 × 8
Linear 8 → 32
4 × 8 → 4 × 32
GELU
4 × 32
Dropout (ระหว่างฝึกเท่านั้น)
4 × 32
Linear 32 → 8
4 × 32 → 4 × 8
MLP output 4 × 8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
$U = X_0 + \text{Dropout}(\text{MHA}(LN_1(X_0)))$
$Y = U + \text{Dropout}(\text{MLP}(LN_2(U)))$
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

Positional Encoding คืออะไร

Positional Encoding ทำหน้าที่ช่วยบอกโมเดลว่าอะไรอยู่ตรงไหน เนื่องจากคุณลักษณะ (Feature Vector) สามารถบอกได้ว่า Image Patch “มีอะไร” แต่ไม่มีข้อมูลช่วยบอกตำแหน่ง ที่ช่วยบอกว่า “อยู่ตรงไหน”

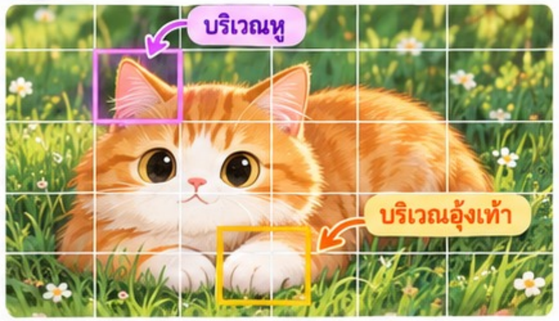
1. แนวคิดจากลำดับคำใน Transformer



คำเดิม แต่ลำดับต่างกัน ความหมายเปลี่ยน

- Self-attention แบบไม่มีสัญญาณตำแหน่ง ไม่รู้ลำดับโดยตรง
- Transformer ปี 2017 เพิ่มเวกเตอร์ตำแหน่งแบบ sine/cosine

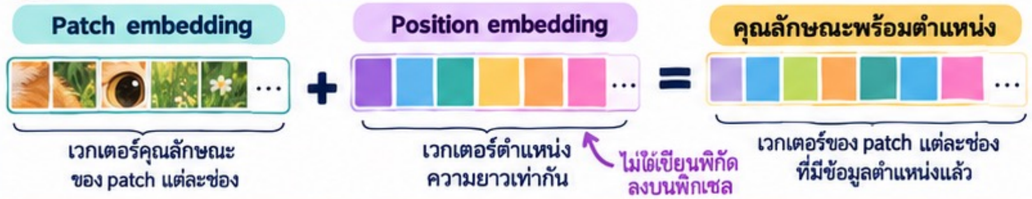
2. ในภาพ: ตำแหน่งของแต่ละ patch





- รู้ว่ามีหู ตา และอุ้งเท้า **ยังไม่เท่ากับรู้ว่าอยู่ตรงไหน**
- **เพิ่มข้อมูลตำแหน่ง** เพื่อช่วยเรียนรู้โครงสร้างเชิงพื้นที่

3. ViT เดิมใส่ตำแหน่งอย่างไร?



- **บวก**เวกเตอร์ตำแหน่งเข้ากับเวกเตอร์คุณลักษณะ
- ViT เดิมใช้เวกเตอร์ตำแหน่ง 1D ที่เรียนรู้ได้
- แต่ละลำดับ token สอดคล้องกับช่อง patch ในภาพ

4. ยังจำเป็นไหม?

สำหรับ ViT มาตรฐาน: ควรมี

หากไม่มีสัญญาณตำแหน่งอื่น
โมเดลจะไม่รู้การจัดวาง patches โดยตรง



ไม่จำเป็นต้องใช้วิธีบวกเวกเตอร์เสมอไป

อาจใช้ตำแหน่งสัมพัทธ์ หรือโครงสร้างที่รับรู้พื้นที่แทน



การตัดออกควรทดลองตามสถาปัตยกรรมและงาน

ไม่ใช่แค่ “มีอะไรในภาพ” แต่ต้องช่วยให้รู้ว่า “แต่ละส่วนอยู่สัมพันธ์กันอย่างไร”

Image generated by OpenAI (2026).

Positional Encoding คืออะไร

Positional Encoding แบ่งได้ทั้งตามวิธีสร้าง ตำแหน่งที่ใส่ และชนิดข้อมูล ซึ่งสามารถเรียกรวม ๆ ว่า วิธีใส่ข้อมูลตำแหน่ง เพราะบางวิธีปรับคะแนน Attention หรือหมุน Q/K แทนการบวก embedding



Image generated by OpenAI (2026).

Positional Encoding คืออะไร

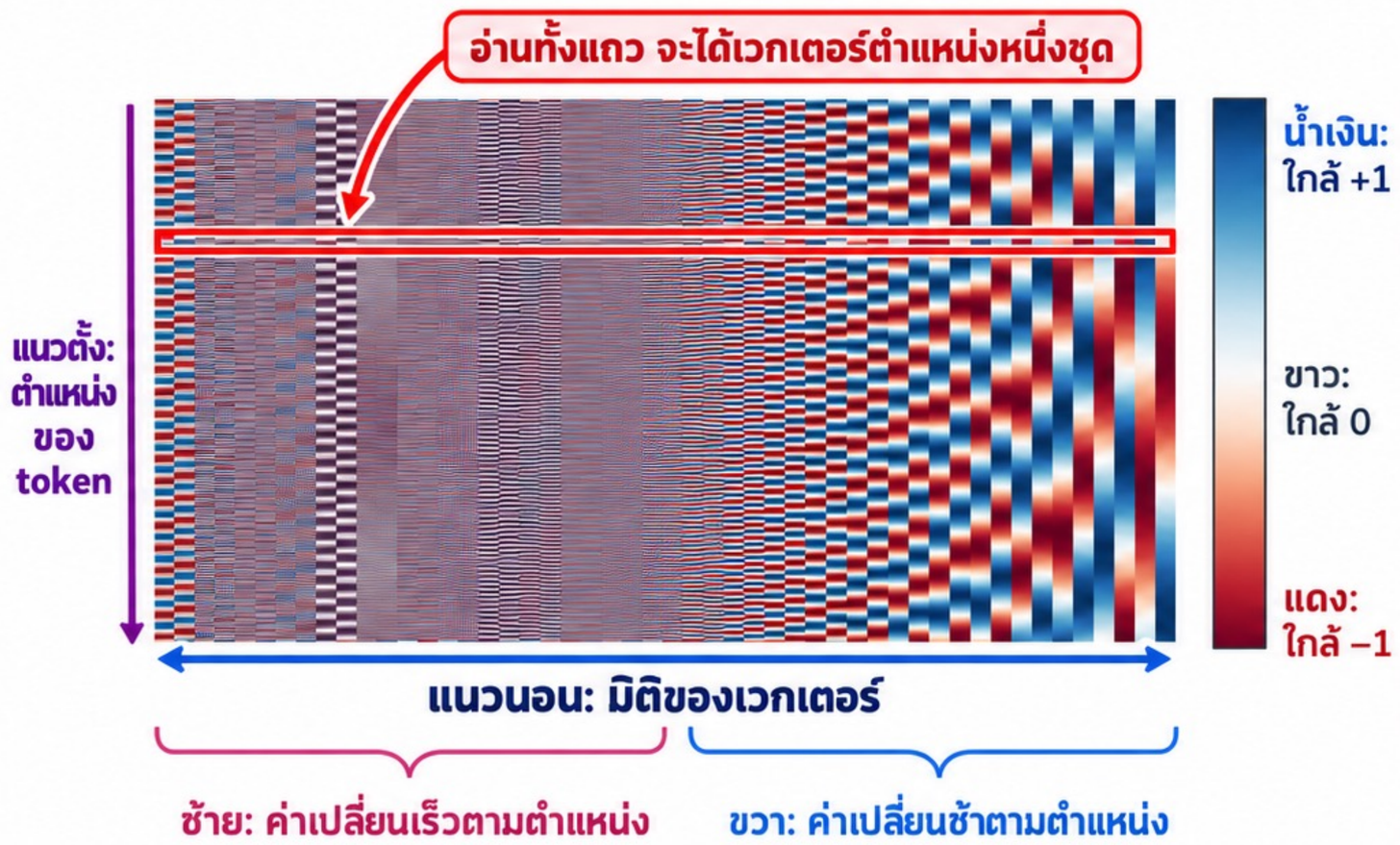
Positional Encoding แบ่งได้ทั้งตามวิธีสร้าง ตำแหน่งที่ใส่ และชนิดข้อมูล ซึ่งสามารถเรียกรวม ๆ ว่า วิธีใส่ข้อมูลตำแหน่ง เพราะบางวิธีปรับคะแนน Attention หรือหมุน Q/K แทนการบวก embedding



Image generated by OpenAI (2026).

Positional Encoding แบบ Fixed Sinusoidal

Positional Encoding แบบ Fixed Sinusoidal ช่วยให้โมเดลรับรู้ตำแหน่งของข้อมูล โดยใช้ฟังก์ชัน Sine และ Cosine คำนวณรหัสตำแหน่งจากสูตรที่กำหนดไว้ ตำแหน่งเดิมจึงได้รับรหัสเดิมเสมอ โดยไม่ต้องเรียนรู้รหัสตำแหน่งจากการฝึกสอน



1. แต่ละตำแหน่งมีรหัสของตัวเอง

- ตำแหน่งต่างกัน ได้รูปแบบตัวเลขต่างกัน
- อ่านตามแถว ไม่ใช่ตามคอลัมน์

2. ใช้ sine/cosine หลายความถี่

- แต่ละคู่มิติใช้ sine และ cosine
- ความถี่ต่างกัน ทำให้ลายถี่และลายห่าง



3. นำรหัสไปบวกกับคุณลักษณะ

Feature + Position = Input

- บวกทีละช่อง ความยาวเวกเตอร์เท่าเดิม

Image generated by OpenAI (2026).

Positional Encoding แบบ Fixed Sinusoidal

Positional Encoding แบบ Fixed Sinusoidal ช่วยให้โมเดลรับรู้ตำแหน่งของข้อมูล โดยใช้ฟังก์ชัน Sine และ Cosine คำนวณรหัสตำแหน่งจากสูตรที่กำหนดไว้ ตำแหน่งเดิมจึงได้รับรหัสเดิมเสมอ โดยไม่ต้องเรียนรู้รหัสตำแหน่งจากการฝึกสอน

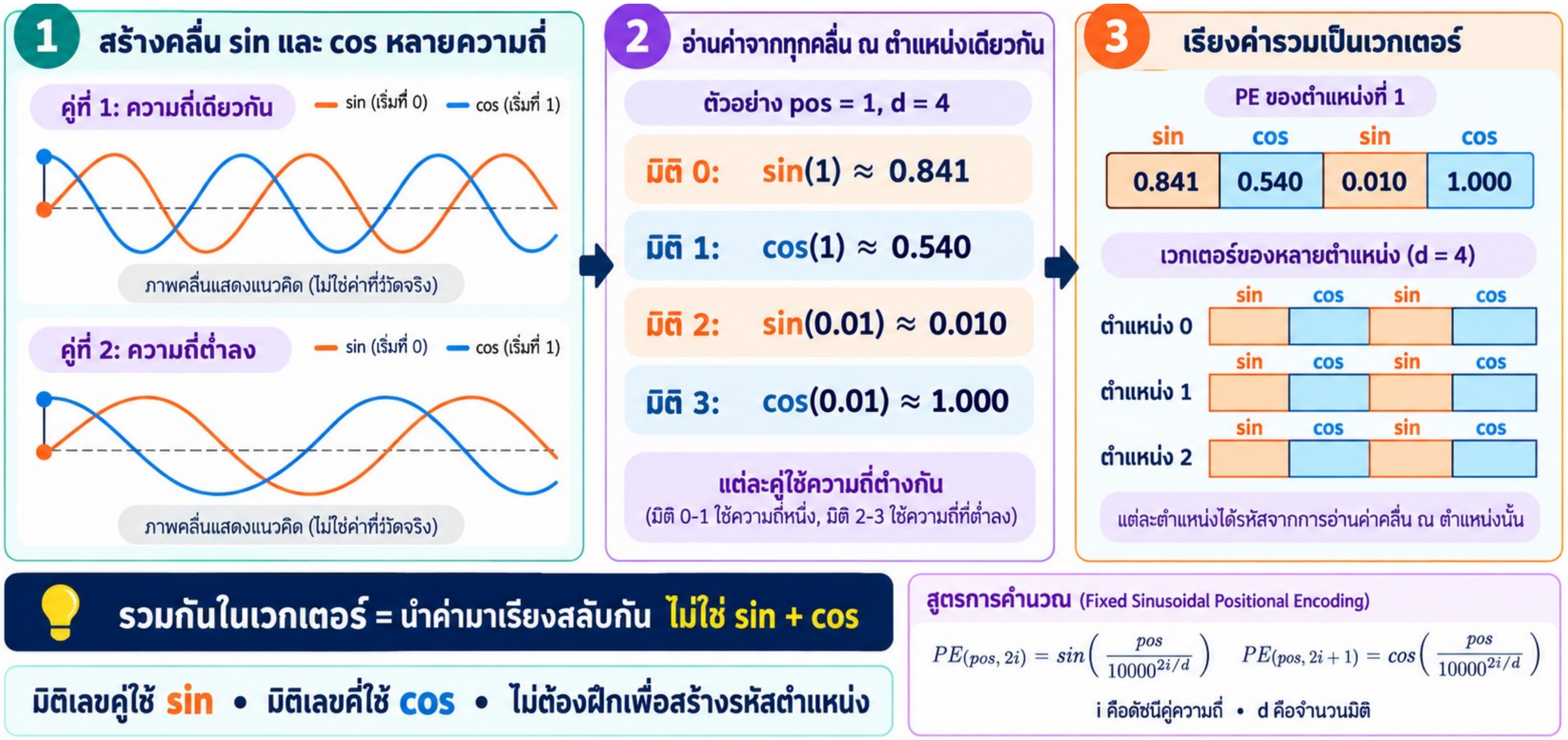


Image generated by OpenAI (2026).

Positional Encoding แบบ Learned Absolute

Positional Encoding แบบ Learned Absolute ช่วยให้โมเดลรับรู้ตำแหน่งของข้อมูล โดยกำหนดเวกเตอร์ที่เรียนรู้ได้ให้แต่ละตำแหน่ง และปรับค่าเวกเตอร์ระหว่างการฝึกสอน เมื่อฝึกเสร็จแล้ว ตำแหน่งเดิมจะใช้เวกเตอร์เดียวกันในทุกข้อมูลที่ป้อนเข้าโมเดล ภายใต้รูปแบบตำแหน่งเดียวกัน

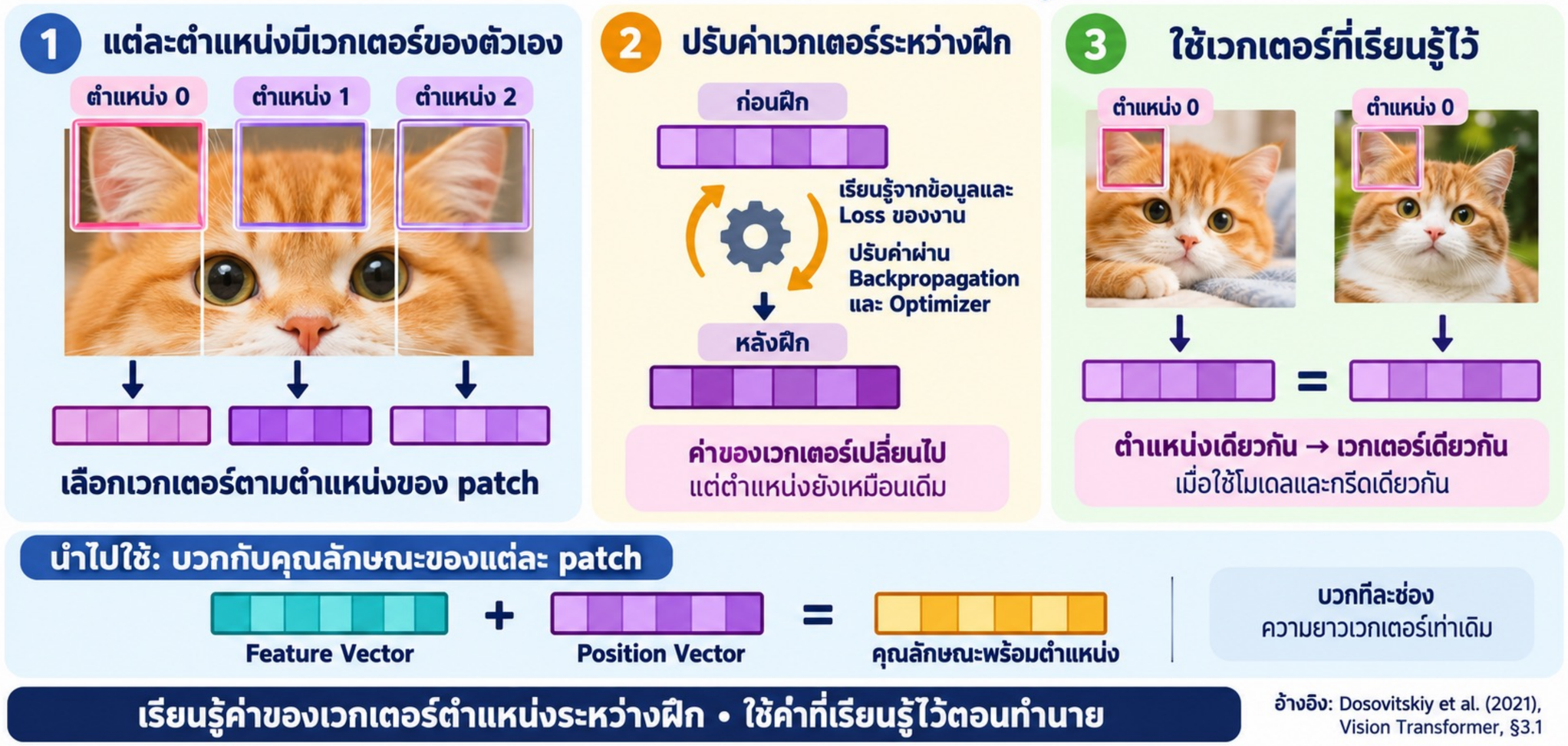


Image generated by OpenAI (2026).

Positional Encoding แบบ Relative Position Bias

Positional Encoding แบบ Relative Position Bias ช่วยให้โมเดลรับรู้ระยะห่างและทิศทางระหว่างข้อมูลแต่ละคู่ โดยเพิ่มค่า Bias ตามตำแหน่งสัมพัทธ์ลงในคะแนน Attention ก่อนผ่าน Softmax ค่า Bias นี้เรียนรู้ระหว่างการฝึกสอน ทำให้โมเดลใช้ความสัมพันธ์เชิงตำแหน่งประกอบการกำหนดน้ำหนัก Attention ได้



Image generated by OpenAI (2026).

เปรียบเทียบความแตกต่าง

Learned Absolute และ Relative Position Bias แบบเรียนรู้ ต่างเรียนรู้พารามิเตอร์เกี่ยวกับตำแหน่งระหว่างการฝึกสอน โดย Learned Absolute เรียนรู้ว่า “Patch อยู่ตรงไหนในภาพ” แล้วบวกเวกเตอร์ตำแหน่งเข้ากับ Patch Embedding ส่วน Relative Position Bias เรียนรู้ว่า “Patch สองอันอยู่ห่างกันเท่าไรและอยู่ทิศทางใด” แล้วเพิ่มค่า Bias ลงในคะแนน Attention ก่อนผ่าน Softmax

ประเด็น	Learned Absolute	Relative Position Bias แบบเรียนรู้
เรียนรู้อะไร	เรียนรู้เวกเตอร์สำหรับบ่งบอกข้อมูลตำแหน่งแต่ละจุด	เรียนรู้ค่า Bias สำหรับบ่งบอกระยะและทิศทางระหว่างคู่ Patch
นำไปใช้ตรงไหน	บวกกับ Patch Embedding ก่อนเข้า Transformer	บวกกับ คะแนน Attention ก่อน Softmax
สิ่งที่ใช้ร่วมกัน	Patch ที่อยู่ตำแหน่งเดียวกันในภาพต่าง ๆ ใช้เวกเตอร์ตำแหน่งเดียวกัน	คู่ Patch ที่มีระยะและทิศทางเดียวกัน ใช้ค่า Bias เดียวกันภายในตารางของ Head นั้น

Image generated by OpenAI (2026).

Positional Encoding แบบ Conditional Positional Encoding

Positional Encoding แบบ Conditional Positional Encoding ช่วยให้โมเดลรับรู้ตำแหน่งของข้อมูล โดยสร้างรหัสตำแหน่งจากคุณลักษณะของแต่ละ Patch และ Patch รอบข้าง ผ่านโครงข่ายที่เรียนรู้ระหว่างการฝึกสอน แล้วบวกรหัสที่ได้เข้ากับคุณลักษณะเดิม รหัสตำแหน่งจึงปรับเปลี่ยนตามข้อมูลภาพที่ป้อนเข้า แม้จะเป็นตำแหน่งเดียวกันก็ตาม

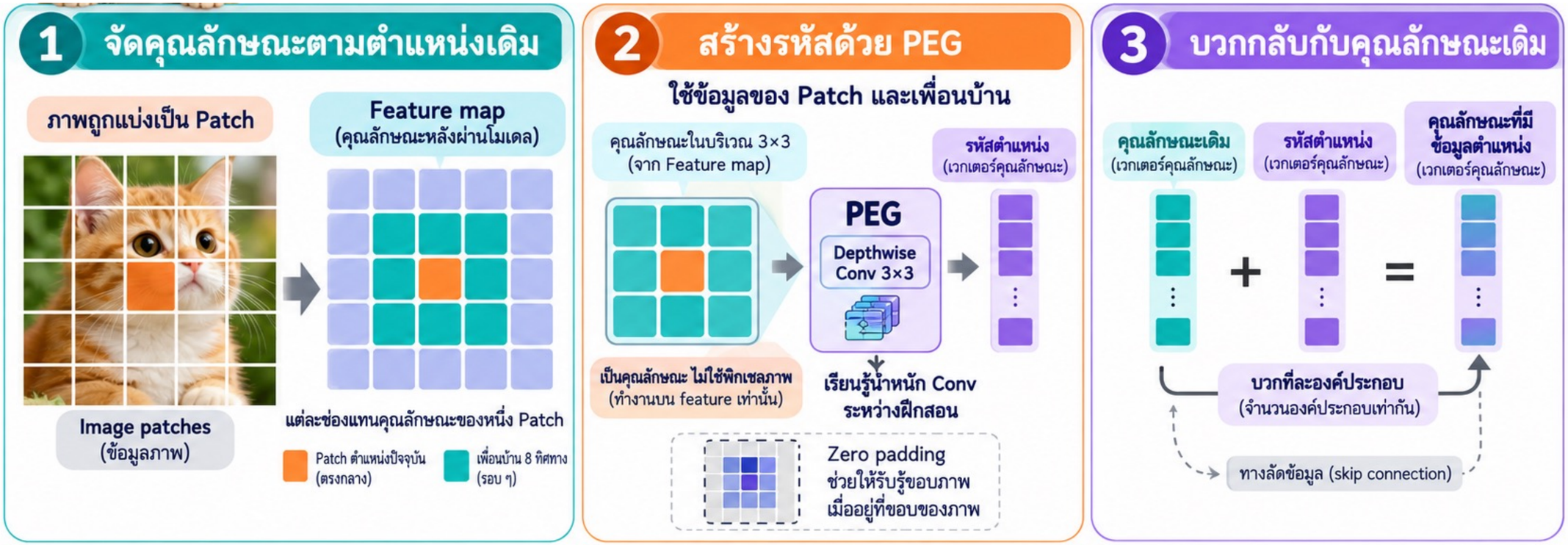
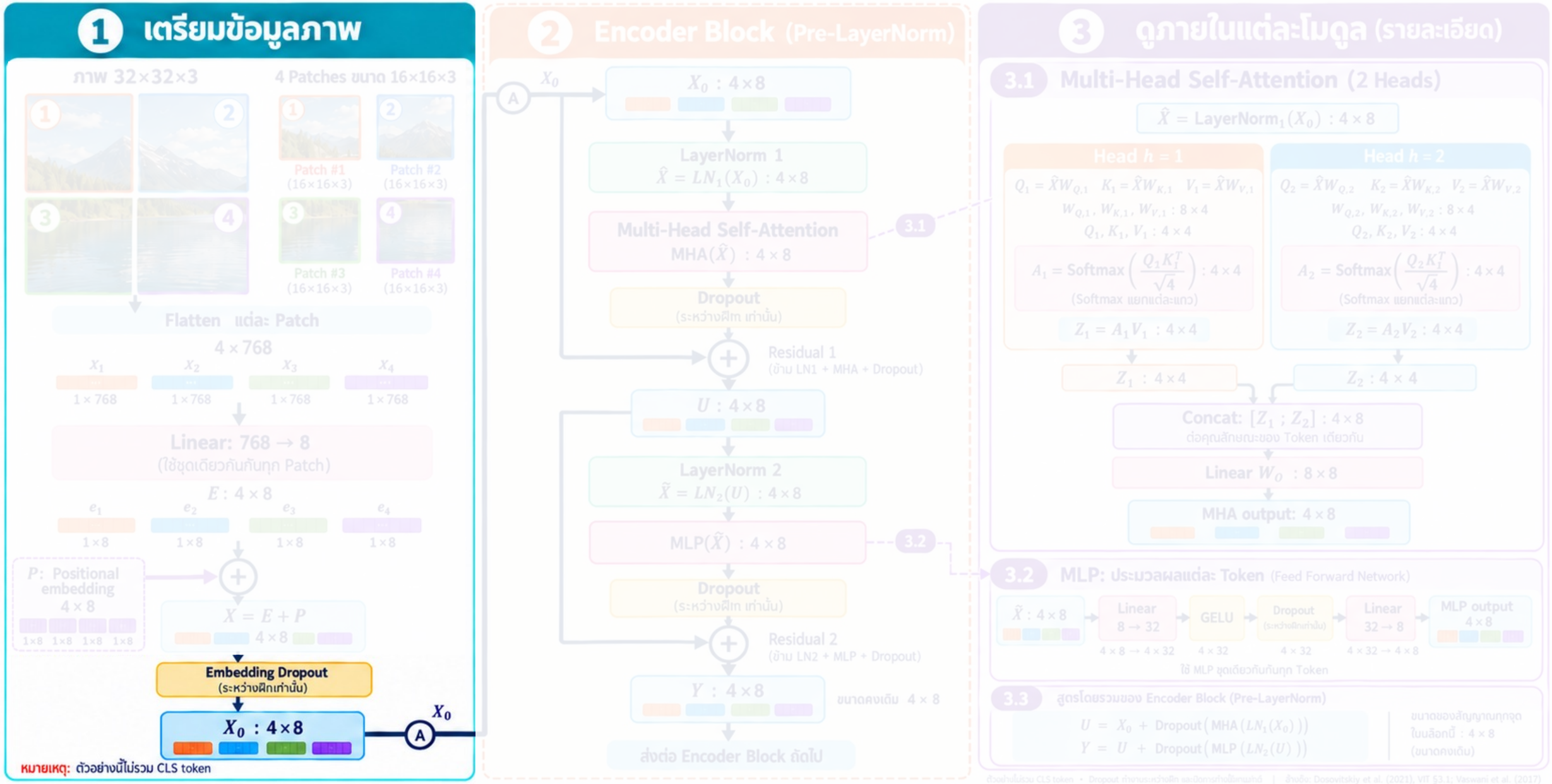
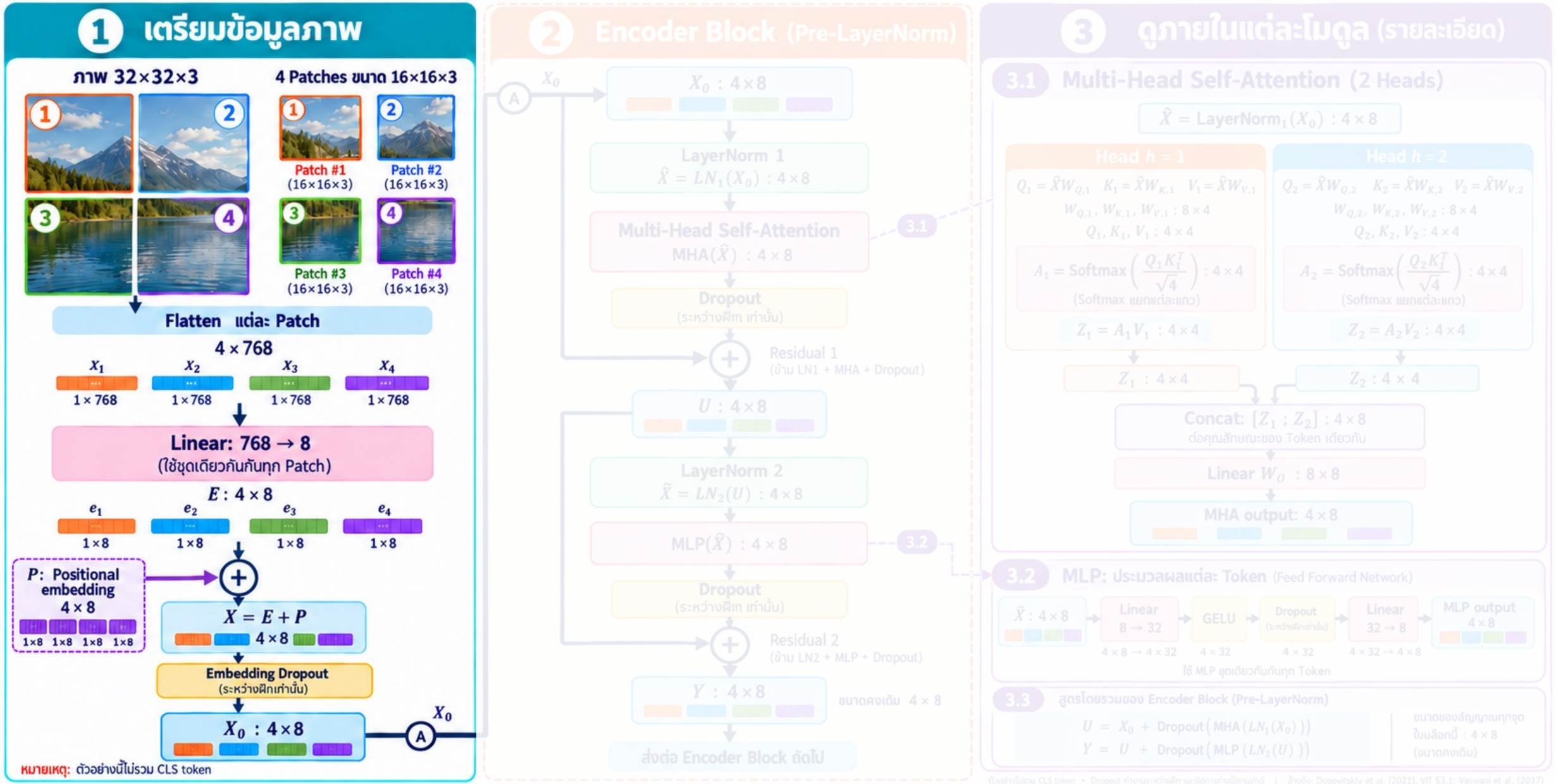
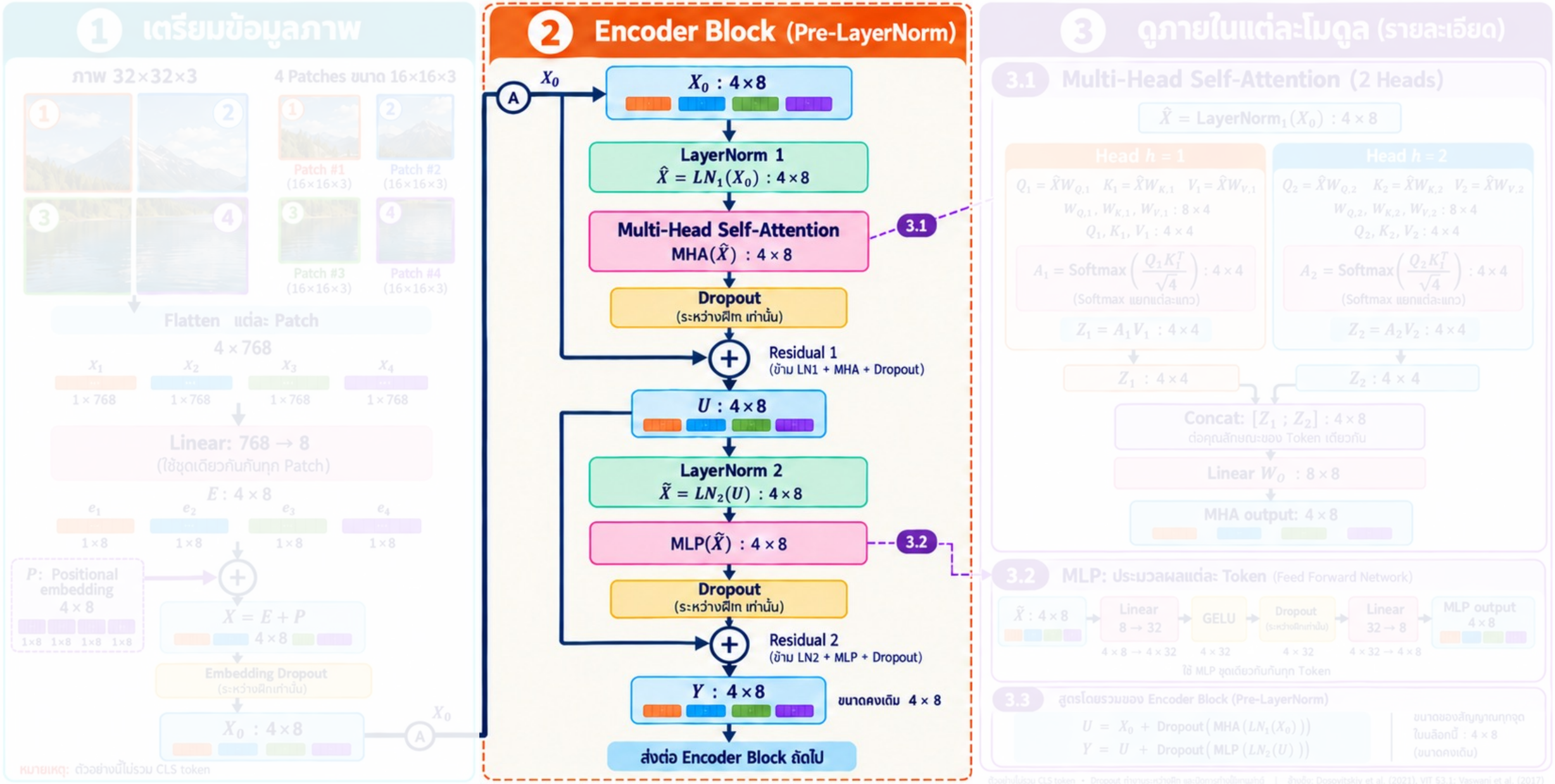


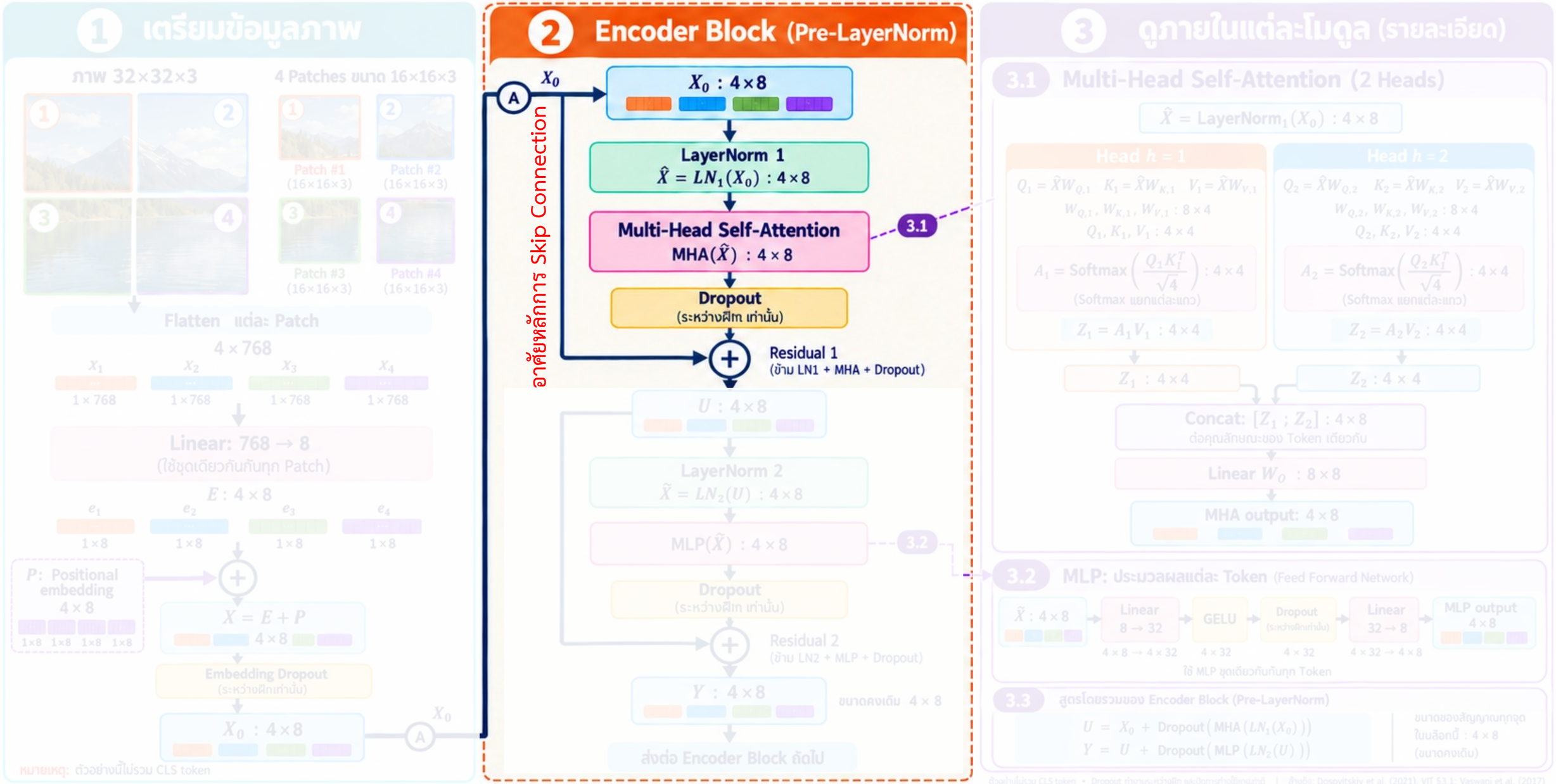
Image generated by OpenAI (2026).

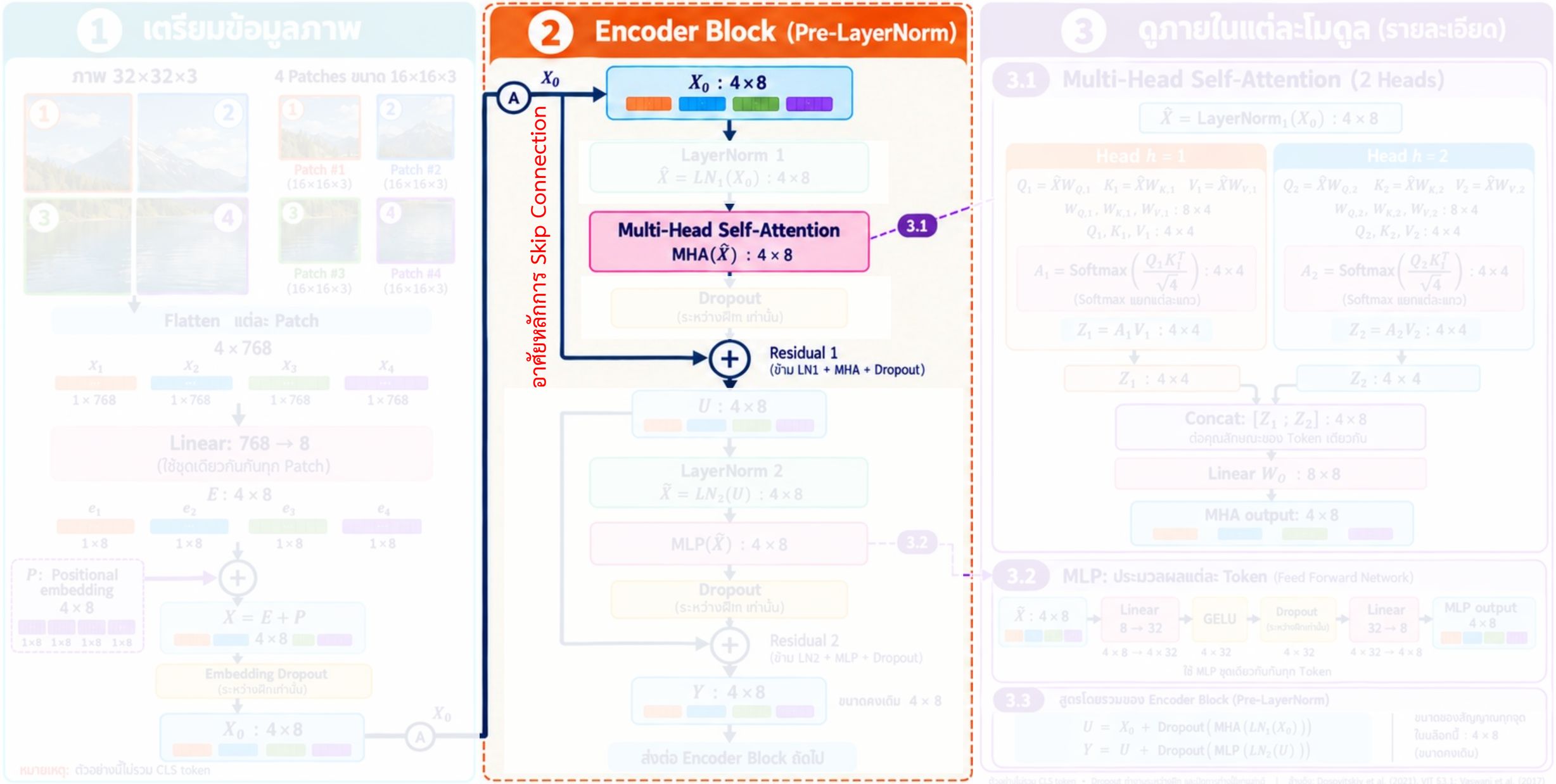

1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
x_1 x_2 x_3 x_4
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
e_1 e_2 e_3 e_4
1×8
P: Positional embedding
4 × 8
1×8 1×8 1×8 1×8
$X = E + P$
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X_0 : 4×8
X_0
A
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X_0
A
X_0 : 4×8
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4×8
Multi-Head Self-Attention
MHA($\hat{X}$) : 4×8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4×8
MLP($\tilde{X}$) : 4×8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
$\hat{X} = \text{LayerNorm}_1(X_0)$: 4×8
Head $h = 1$
$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$
$W_{Q,1}, W_{K,1}, W_{V,1}$: 8×4
Q_1, K_1, V_1 : 4×4
$A_1 = \text{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right)$: 4×4
(Softmax แยกแต่ละแถว)
$Z_1 = A_1 V_1$: 4×4
Head $h = 2$
$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$
$W_{Q,2}, W_{K,2}, W_{V,2}$: 8×4
Q_2, K_2, V_2 : 4×4
$A_2 = \text{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right)$: 4×4
(Softmax แยกแต่ละแถว)
$Z_2 = A_2 V_2$: 4×4
Z_1 : 4×4
Z_2 : 4×4
Concat: $[Z_1 ; Z_2]$: 4×8
ต่อคุณลักษณะของ Token เดียวกัน
Linear W_O : 8×8
MHA output: 4×8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
$\tilde{X}$: 4×8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4×8
4×8 → 4×32
4×32
4×32
4×32 → 4×8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
$U = X_0 + \text{Dropout}(\text{MHA}(LN_1(X_0)))$
$Y = U + \text{Dropout}(\text{MLP}(LN_2(U)))$
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

ขั้นตอนการเตรียมข้อมูลภาพ หลายสถาปัตยกรรมจะดำเนินการเพียงครั้งเดียว




1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4×8
3.1
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4×8
3.2
Dropout
(ระหว่างฝึกเท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4×8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8×4
Q1, K1, V1 : 4×4
A1 = Softmax(Q1K1T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4×4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8×4
Q2, K2, V2 : 4×4
A2 = Softmax(Q2K2T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4×4
Z1 : 4×4
Z2 : 4×4
Concat: [Z1 ; Z2] : 4×8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8×8
MHA output: 4×8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4×8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4×8
4×8 → 4×32
4×32
4×32
4×32 → 4×8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X0
A
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4 × 8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
อาศัยหลักการ Skip Connection
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4 × 8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4 × 8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8 × 4
Q1, K1, V1 : 4 × 4
A1 = Softmax(Q1K1T/√4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4 × 4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8 × 4
Q2, K2, V2 : 4 × 4
A2 = Softmax(Q2K2T/√4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4 × 4
Z1 : 4 × 4
Z2 : 4 × 4
Concat: [Z1 ; Z2] : 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4 × 8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4 × 8
4 × 8 → 4 × 32
4 × 32
4 × 32
4 × 32 → 4 × 8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X0
A
X0 : 4×8
อาศัยหลักการ Skip Connection
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4 × 8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4 × 8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4 × 8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8 × 4
Q1, K1, V1 : 4 × 4
A1 = Softmax(Q1K1T/√4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4 × 4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8 × 4
Q2, K2, V2 : 4 × 4
A2 = Softmax(Q2K2T/√4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4 × 4
Z1 : 4 × 4
Z2 : 4 × 4
Concat: [Z1 ; Z2] : 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4 × 8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4 × 8
4 × 8 → 4 × 32
4 × 32
4 × 32
4 × 32 → 4 × 8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างนี้ไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อใช้งานจริง | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

Attention และ Self-attention ในงานภาพ

Self-attention เป็นรูปแบบหนึ่งของ Attention

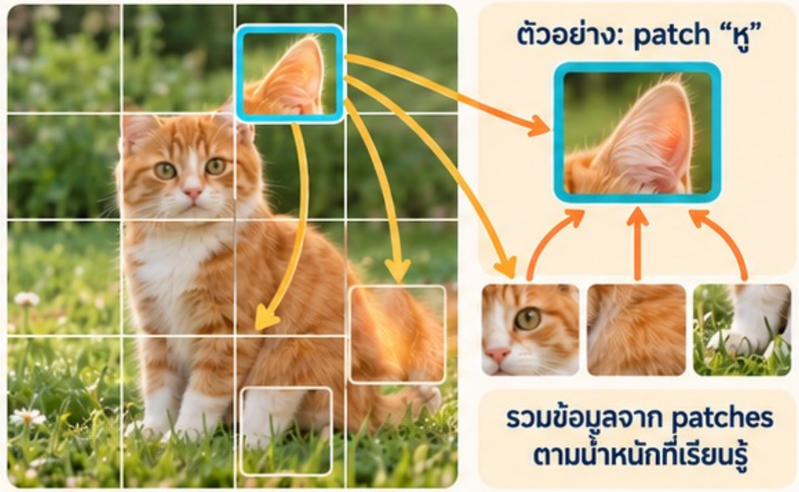
1 Attention: ให้น้ำหนักกับข้อมูล



- เรียนรู้ว่าข้อมูลส่วนใดควรมีบทบาทมากขึ้น
- อาจให้น้ำหนักตำแหน่ง ช่องคุณลักษณะ หรือความสัมพันธ์

2 2. Self-attention

เรียนรู้ความสัมพันธ์ระหว่าง patches ในภาพเดียวกัน



- แต่ละ patch เรียนรู้ว่าควรใช้ข้อมูลจาก patch ใด มากน้อยเพียงใด
- รวมข้อมูลตามน้ำหนัก เพื่อสร้างคุณลักษณะที่มีบริบท
- Q, K, V มาจากชุดคุณลักษณะเดียวกัน

3 Cross-attention: เชื่อมข้อมูลต่างชุด



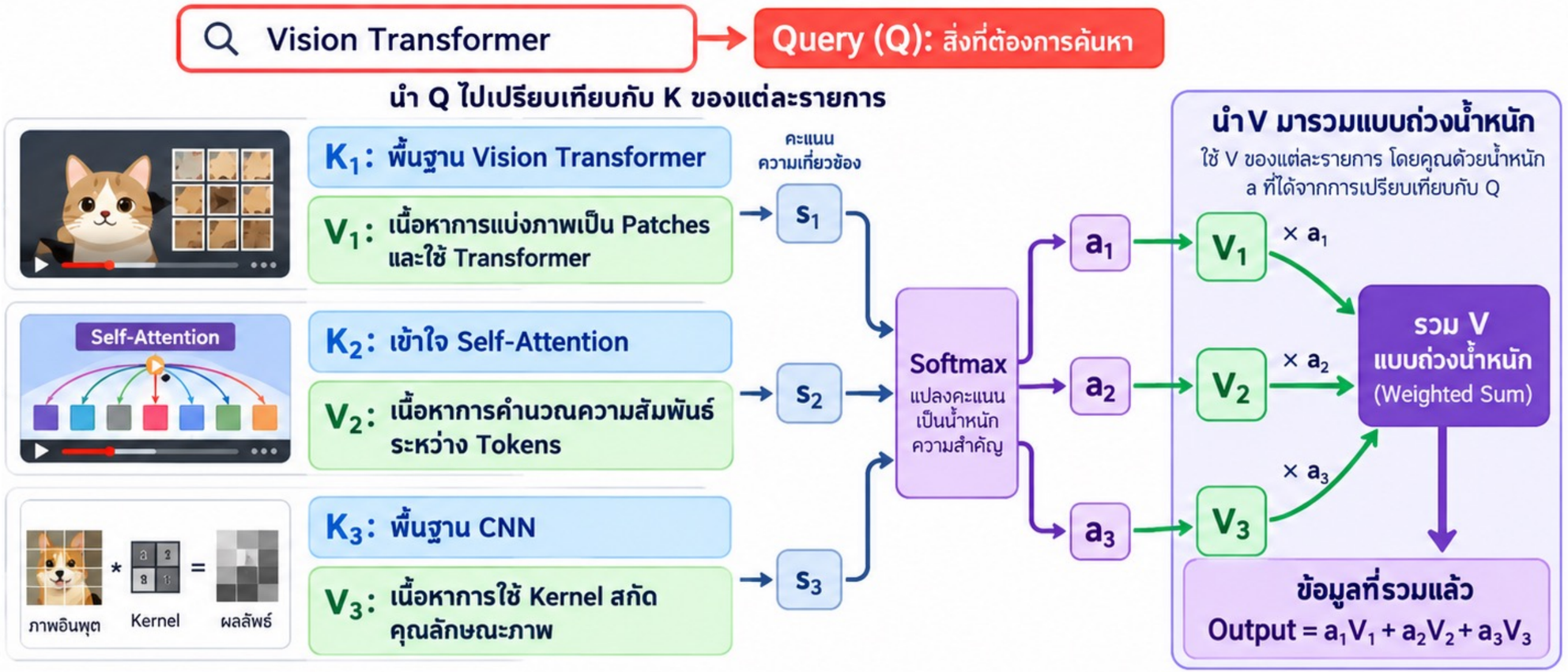
- ตัวอย่าง: ข้อความเชื่อมกับคุณลักษณะภาพ
- Q มาจากข้อความ; K, V มาจากภาพ

Self-attention = เรียนรู้ความสัมพันธ์ + รวมข้อมูลภายในชุดเดียวกัน

อ้างอิง: Vaswani et al., NeurIPS 2017 | Dosovitskiy et al., ICLR 2021 | Woo et al., ECCV 2018

Image generated by OpenAI (2026).

Q, K, V ใน Self-attention คืออะไร



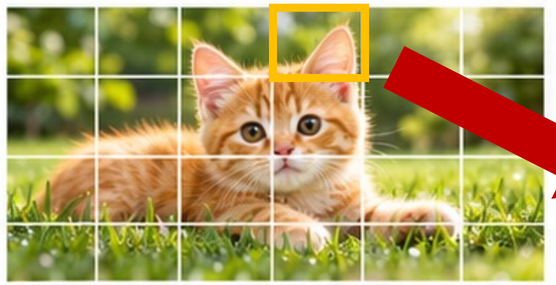
Q: ต้องการข้อมูลอะไร?
คือสิ่งที่เราสนใจค้นหา เช่น “Vision Transformer”

K: ใช้เทียบว่าข้อมูลเกี่ยวข้องแค่ไหน?
คือข้อความ/คุณลักษณะของแต่ละรายการ ที่นำมาเปรียบเทียบกับ Q เพื่อคำนวณคะแนนความเกี่ยวข้อง

V: ข้อมูลที่จะนำมารวม
คือเนื้อหาจริงของแต่ละรายการ เช่น เนื้อหาของวิดีโอ (แนวคิด รูปภาพ เสียง ฯลฯ)

Image generated by OpenAI (2026).

Q, K, V ใน Self-attention คืออะไร ในงานภาพ



ภาพ

มาจากคุณลักษณะชุดเดียวกัน แต่เรียนรู้การแปลงแยกกัน





patches

X คือคุณลักษณะของ tokens ที่เข้าสู่ชั้นนี้



เวกเตอร์คุณลักษณะ X

Q — Query
ใช้ค้นหาข้อมูลที่เกี่ยวข้อง



ฉันควรใช้ข้อมูลจากใคร?

- สร้างจาก X ผ่านการแปลง W_Q
- เปรียบเทียบกับ K ของแต่ละ patch

Q ใช้สำหรับคำนวณน้ำหนัก Attention
โดยค้นหาว่า patch อื่น ๆ ที่เกี่ยวข้องคืออะไร

K — Key
ใช้เป็นตัวแทนสำหรับเปรียบเทียบ



ฉันมีลักษณะที่เกี่ยวข้องแค่ไหน?

ลักษณะสีและลวดลาย | ลักษณะพื้นผิวขน | ลักษณะของพื้นหญ้า

- สร้างจาก X ผ่านการแปลง W_K
- ใช้จับคู่กับ Q เพื่อคำนวณคะแนน

K ทำหน้าที่เป็นตัวแทนสำหรับเปรียบเทียบ
เพื่อให้ Q คำนวณว่าสอดคล้องกันมากแค่ไหน

V — Value
ข้อมูลที่จะนำมารวม



นี่คือข้อมูลที่ฉันส่งให้

- สร้างจาก X ผ่านการแปลง W_V
- นำมารวมตามน้ำหนัก Attention

V คือข้อมูลของแต่ละ patch
ที่จะถูกนำมารวมกันตามน้ำหนัก Attention

เปรียบเทียบ Q กับ K → **แปลงคะแนนเป็นน้ำหนักด้วย softmax** → **รวม V ตามน้ำหนัก**

ทุก patch มีทั้ง Q, K และ V — ไม่ได้แบ่ง patch ให้ทำคนละหน้าที่

Image generated by OpenAI (2026).

แนวคิดการสกัด Q, K, V จากข้อมูล

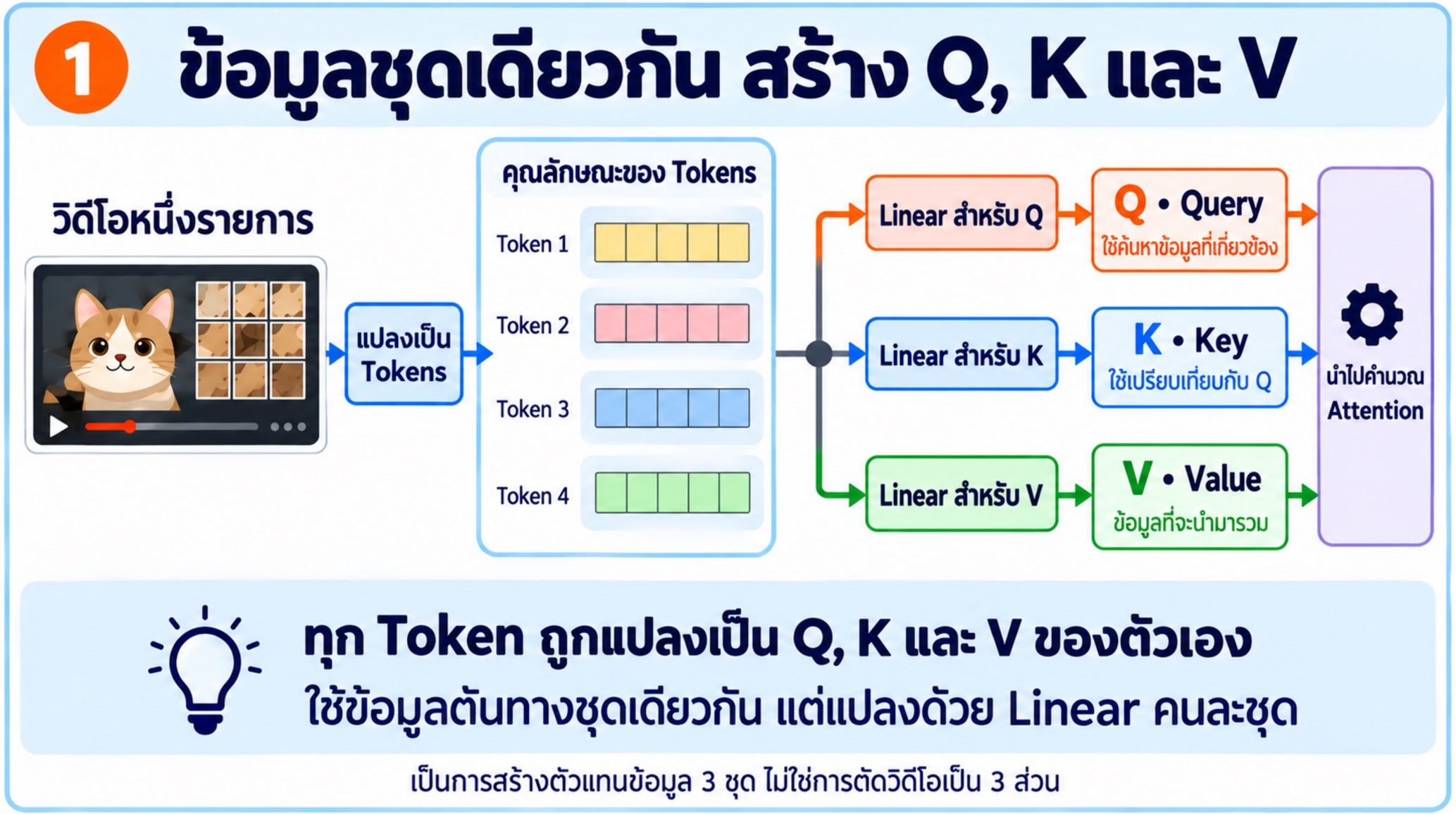
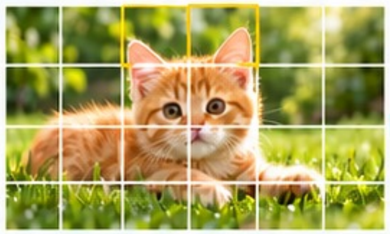


Image generated by OpenAI (2026).

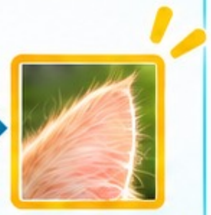
ขั้นตอนการสกัด Q, K, V จากภาพ

1. Image Patches

แบ่งภาพเป็นส่วนย่อย



ภาพ



patch (ตัวอย่าง)

แต่ละ patch คือข้อมูลพิกเซลส่วนหนึ่งของภาพ

2. Flatten

เรียงค่าพิกเซลของแต่ละ patch เป็นเวกเตอร์



patch



เวกเตอร์พิกเซล
(พิกเซลทั้งหมดของ patch)

เปลี่ยนรูปแบบข้อมูล
ยังไม่เรียนรู้คุณลักษณะ

3. Patch Embedding

แปลงเวกเตอร์พิกเซลเป็นเวกเตอร์คุณลักษณะ



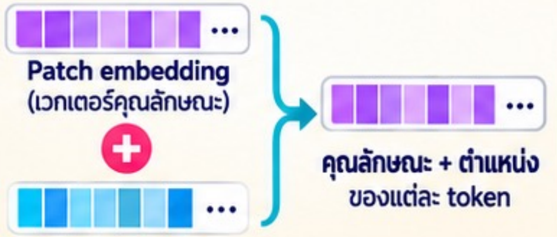
เวกเตอร์พิกเซล
(จากขั้น Flatten)

เวกเตอร์คุณลักษณะ
ของ patch

ใช้พารามิเตอร์ชุดเดียวกันกับทุก patch

4. Positional Encoding

บวกข้อมูลตำแหน่งให้แต่ละ token



ก่อนสร้าง K, Q, V: ผ่าน LayerNorm ใน ViT มาตรฐาน → X = คุณลักษณะของ tokens หลัง LayerNorm

5. แยกเป็น 3 เส้นทางจาก X ชุดเดียวกัน

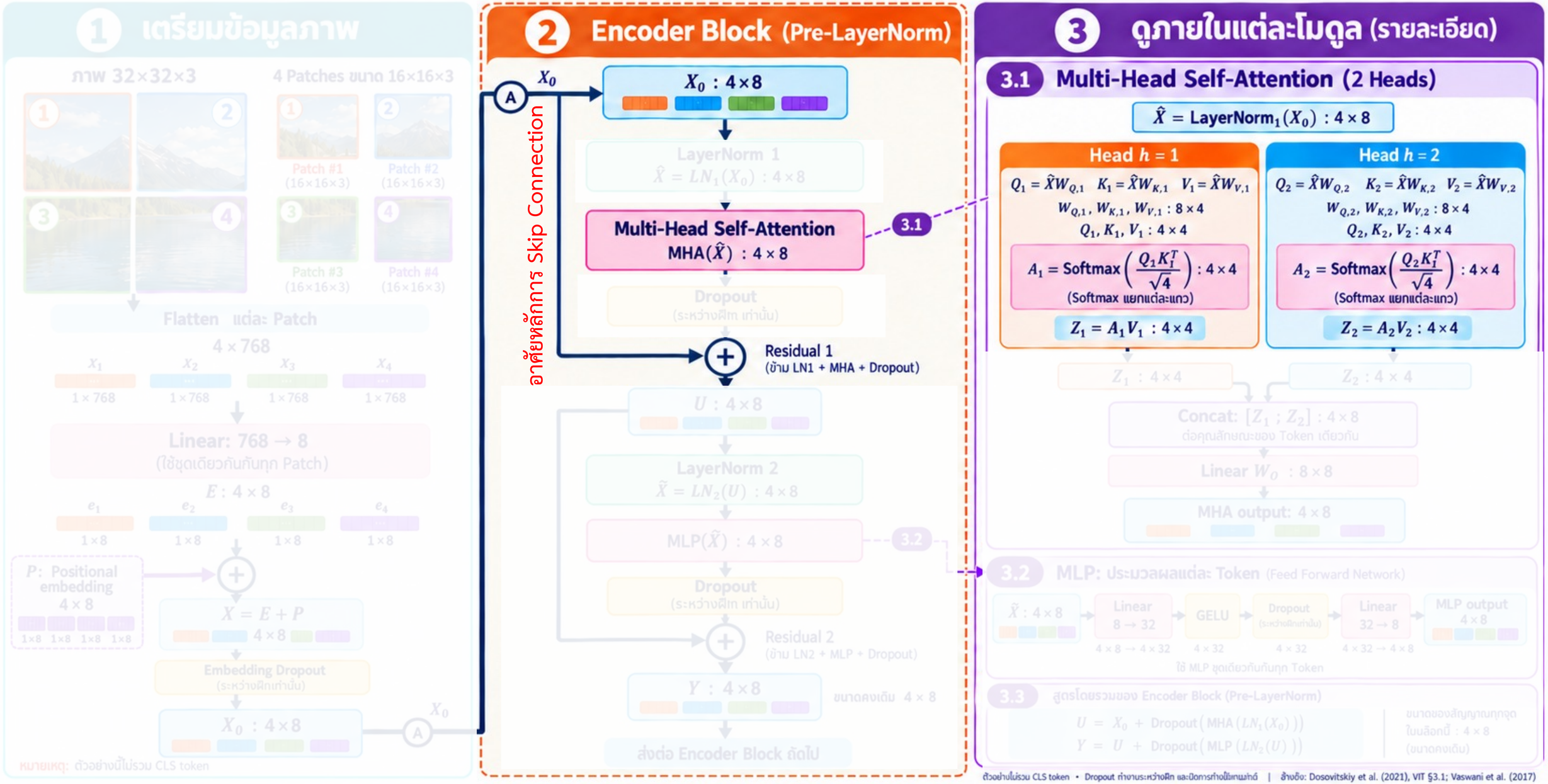
X (คุณลักษณะของ tokens หลัง LayerNorm)	Linear K	K (Key)	ตัวแทนสำหรับเปรียบเทียบกับ Q
X (คุณลักษณะของ tokens หลัง LayerNorm)	Linear Q	Q (Query)	ใช้ค้นหาข้อมูลที่เกี่ยวข้อง
X (คุณลักษณะของ tokens หลัง LayerNorm)	Linear V	V (Value)	ข้อมูลที่จะนำมารวมตามน้ำหนัก Attention

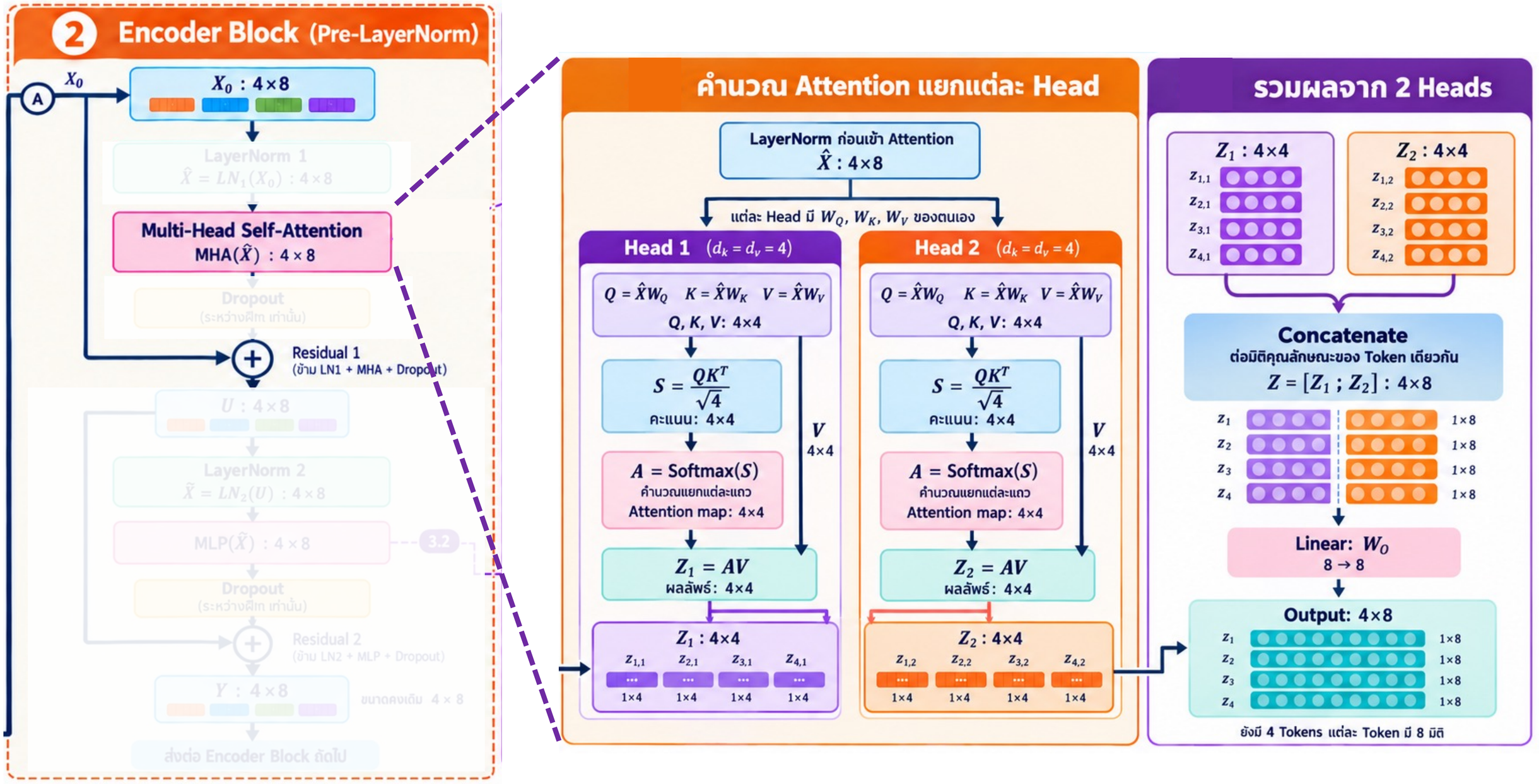
- Linear K, Q, V เรียนรู้พารามิเตอร์แยกกัน
- แต่ละเส้นทางใช้พารามิเตอร์ชุดเดิมกับทุก token



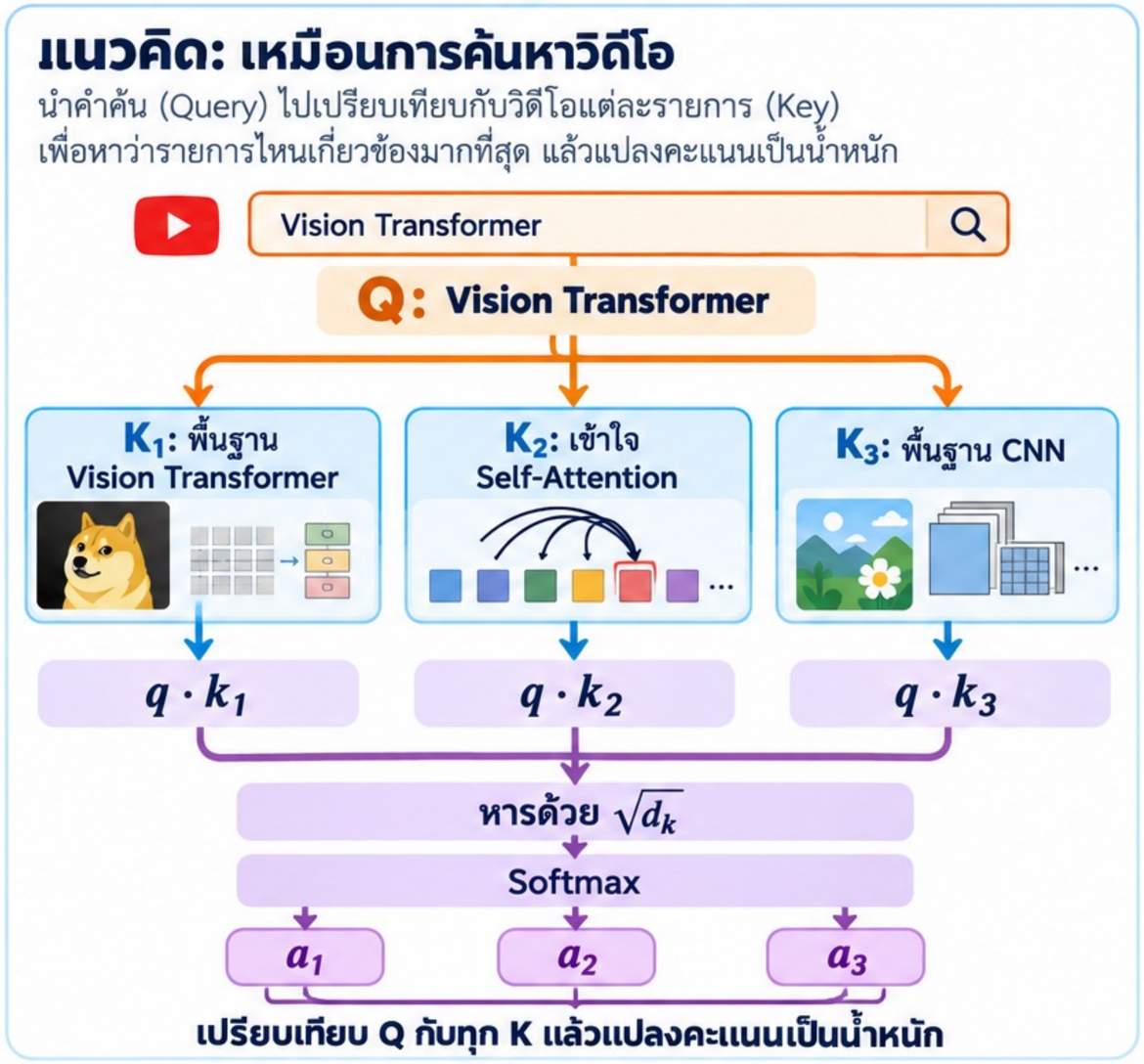
ทุก patch สร้างครบทั้ง K, Q และ V | Q เปรียบเทียบ K เพื่อหาน้ำหนัก แล้วนำ V มารวมตามน้ำหนัก

Image generated by OpenAI (2026).


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
x_1 x_2 x_3 x_4
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
e_1 e_2 e_3 e_4
1×8
P: Positional embedding
4 × 8
$X = E + P$
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X_0 : 4×8
X_0
A
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X_0
X_0 : 4×8
อาศัยหลักการ Skip Connection
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4×8
Multi-Head Self-Attention
MHA($\hat{X}$) : 4×8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4×8
MLP($\tilde{X}$) : 4×8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
$\hat{X} = \text{LayerNorm}_1(X_0)$: 4 × 8
Head $h = 1$
$Q_1 = \hat{X}W_{Q,1}$ $K_1 = \hat{X}W_{K,1}$ $V_1 = \hat{X}W_{V,1}$
$W_{Q,1}, W_{K,1}, W_{V,1}$: 8 × 4
Q_1, K_1, V_1 : 4 × 4
$A_1 = \text{Softmax}\left(\frac{Q_1 K_1^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_1 = A_1 V_1$: 4 × 4
Head $h = 2$
$Q_2 = \hat{X}W_{Q,2}$ $K_2 = \hat{X}W_{K,2}$ $V_2 = \hat{X}W_{V,2}$
$W_{Q,2}, W_{K,2}, W_{V,2}$: 8 × 4
Q_2, K_2, V_2 : 4 × 4
$A_2 = \text{Softmax}\left(\frac{Q_2 K_2^T}{\sqrt{4}}\right)$: 4 × 4
(Softmax แยกแต่ละแถว)
$Z_2 = A_2 V_2$: 4 × 4
Z_1 : 4 × 4
Z_2 : 4 × 4
Concat: $[Z_1 ; Z_2]$: 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear W_O : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
$\tilde{X}$: 4 × 8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4 × 8
4 × 8 → 4 × 32
4 × 32
4 × 32
4 × 32 → 4 × 8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
$U = X_0 + \text{Dropout}(\text{MHA}(LN_1(X_0)))$
$Y = U + \text{Dropout}(\text{MLP}(LN_2(U)))$
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานในเวลาทำนาย | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)


2 Encoder Block (Pre-LayerNorm)
A
X_0
X_0 : 4×8
LayerNorm 1
$\hat{X} = LN_1(X_0)$: 4×8
Multi-Head Self-Attention
MHA($\hat{X}$) : 4×8
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
$\tilde{X} = LN_2(U)$: 4×8
MLP($\tilde{X}$) : 4×8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
คำนวณ Attention แยกแต่ละ Head
LayerNorm ก่อนเข้า Attention
$\hat{X}$: 4×8
แต่ละ Head มี W_Q, W_K, W_V ของตนเอง
Head 1 ($d_k = d_v = 4$)
$Q = \hat{X}W_Q$ $K = \hat{X}W_K$ $V = \hat{X}W_V$
Q, K, V: 4×4
$S = \frac{QK^T}{\sqrt{4}}$
คะแนน: 4×4
A = Softmax(S)
คำนวณแยกแต่ละแถว
Attention map: 4×4
$Z_1 = AV$
ผลลัพธ์: 4×4
V 4×4
Z_1 : 4×4
$z_{1,1}$ $z_{2,1}$ $z_{3,1}$ $z_{4,1}$
1×4 1×4 1×4 1×4
Head 2 ($d_k = d_v = 4$)
$Q = \hat{X}W_Q$ $K = \hat{X}W_K$ $V = \hat{X}W_V$
Q, K, V: 4×4
$S = \frac{QK^T}{\sqrt{4}}$
คะแนน: 4×4
A = Softmax(S)
คำนวณแยกแต่ละแถว
Attention map: 4×4
$Z_2 = AV$
ผลลัพธ์: 4×4
V 4×4
Z_2 : 4×4
$z_{1,2}$ $z_{2,2}$ $z_{3,2}$ $z_{4,2}$
1×4 1×4 1×4 1×4
รวมผลจาก 2 Heads
Z_1 : 4×4
$z_{1,1}$
$z_{2,1}$
$z_{3,1}$
$z_{4,1}$
Z_2 : 4×4
$z_{1,2}$
$z_{2,2}$
$z_{3,2}$
$z_{4,2}$
Concatenate
ต่อมิติคุณลักษณะของ Token เดียวกัน
$Z = [Z_1 ; Z_2]$: 4×8
z_1 z_2 z_3 z_4
1×8 1×8 1×8 1×8
Linear: W_O
8 → 8
Output: 4×8
z_1 z_2 z_3 z_4
1×8 1×8 1×8 1×8
ยังมี 4 Tokens แต่ละ Token มี 8 มิติ

Q เปรียบเทียบกับ K อย่างไร



สูตรที่ใช้ใน Attention มาตรฐาน

$$s_i = \frac{q \cdot k_i}{\sqrt{d_k}} \qquad a_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$$

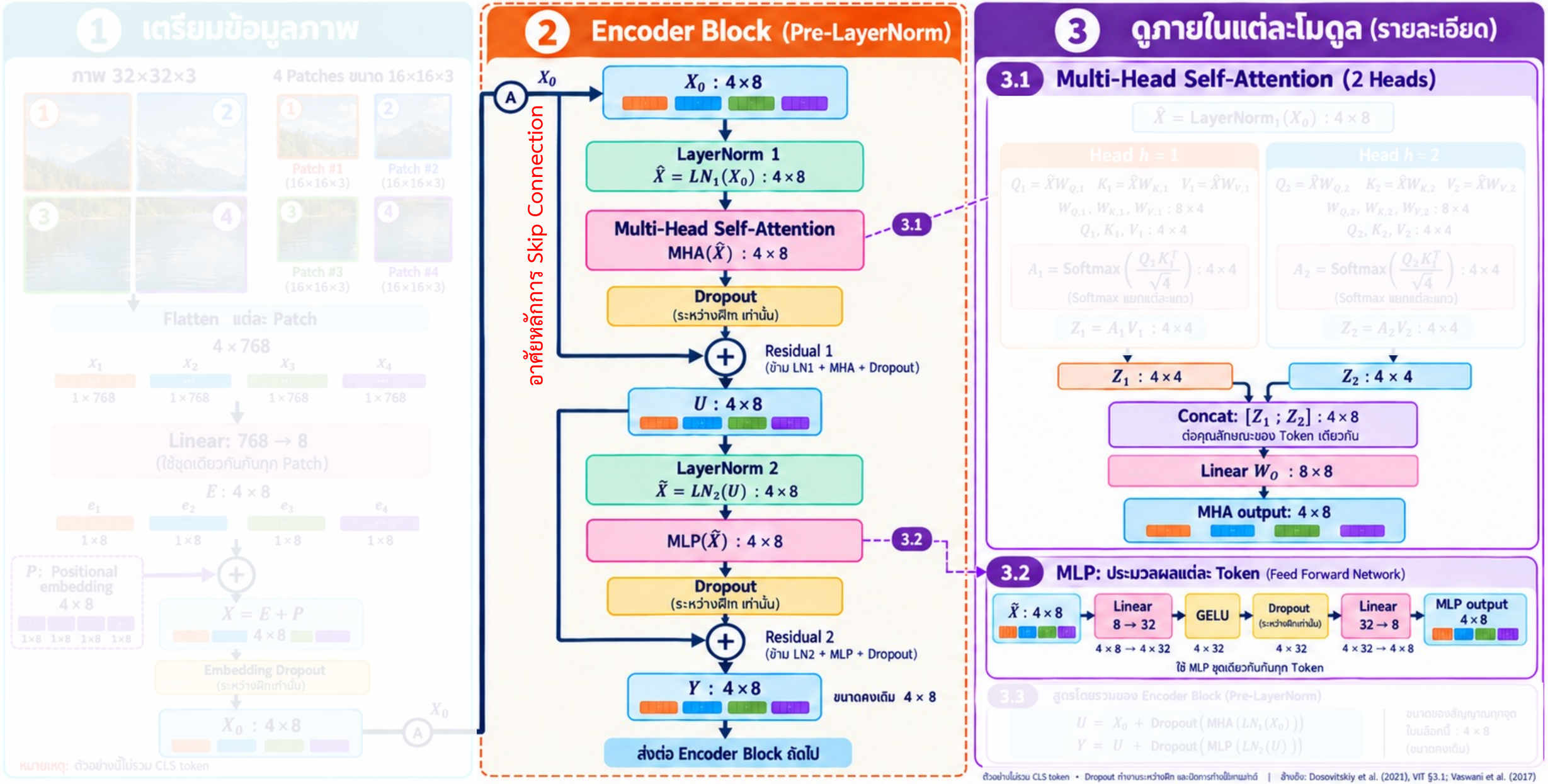
- d_k = จำนวนมิติของ Key ต่อ Head
- คะแนน s_i ไม่จำกัดช่วง -1 ถึง 1
- หลัง Softmax: น้ำหนักรวมกันเป็น 1 $\left(\sum_j a_j = 1\right)$
- เป็นการเปรียบเทียบ Q กับทุก K สำหรับ Query หนึ่งตัว

Image generated by OpenAI (2026).

ความแตกต่างระหว่าง Cosine Similarity และ Scale Dot Product

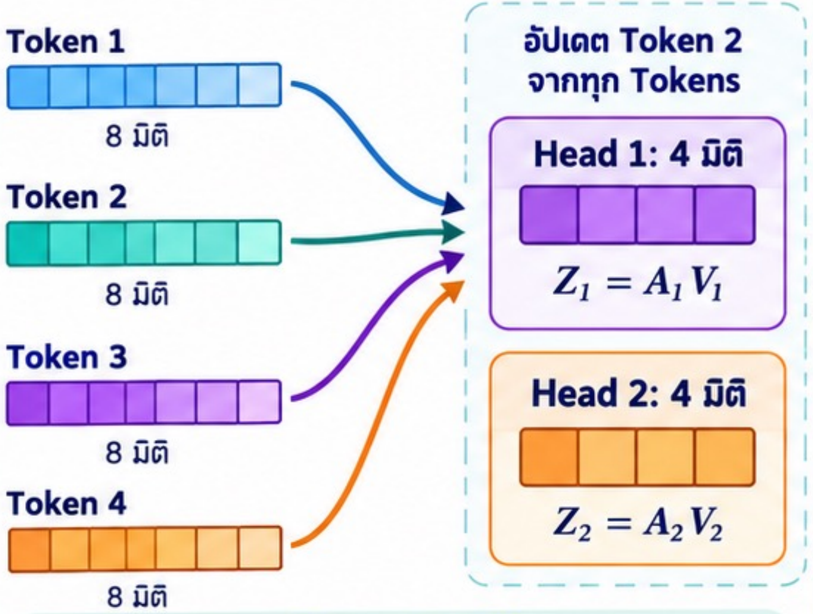
วิธีการ Cosine Similarity และ Scale Dot Product อาศัยหลักการใช้ Dot Product เปรียบเทียบเวกเตอร์ แต่หารด้วยค่าต่างกัน จึงตีความคะแนนต่างกัน

ประเด็น	Cosine Similarity	Scaled Dot-Product
สมการ	$\frac{q \cdot k}{\|q\|\|k\|}$	$\frac{q \cdot k}{\sqrt{d_k}}$
พจน์ที่ใช้หารร	ผลคูณความยาวของเวกเตอร์ทั้งสอง	รากที่สองของจำนวนมิติของ Key
ปัจจัยที่ส่งผลต่อคะแนน	ทิศทาง ของเวกเตอร์	ทิศทางและความยาว ของเวกเตอร์
ช่วงคะแนน	−1 ถึง 1 สำหรับเวกเตอร์ที่ไม่เป็นศูนย์	ไม่มีขอบเขตตายตัว
จุดประสงค์ของการหาร	ตัดอิทธิพลของความยาวเวกเตอร์	ลดขนาดคะแนนเมื่อจำนวนมิติมาก เพื่อช่วยให้ Softmax ไม่กระจุกตัวมากเกินไป


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกันทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
อาศัยหลักการ Skip Connection
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4×8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4×8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4×8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8×4
Q1, K1, V1 : 4×4
A1 = Softmax(Q1K1T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4×4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8×4
Q2, K2, V2 : 4×4
A2 = Softmax(Q2K2T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4×4
Z1 : 4×4
Z2 : 4×4
Concat: [Z1 ; Z2] : 4×8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8×8
MHA output: 4×8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4×8
Linear 8 → 32
4×8 → 4×32
GELU
4×32
Dropout (ระหว่างฝึกเท่านั้น)
4×32
Linear 32 → 8
4×32 → 4×8
MLP output 4×8
ใช้ MLP ชุดเดียวกันกันทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานในเวลาทำนาย | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

1. Attention: รวมข้อมูลข้าม Tokens

แสดงการอัปเดต Token 2 เป็นตัวอย่าง



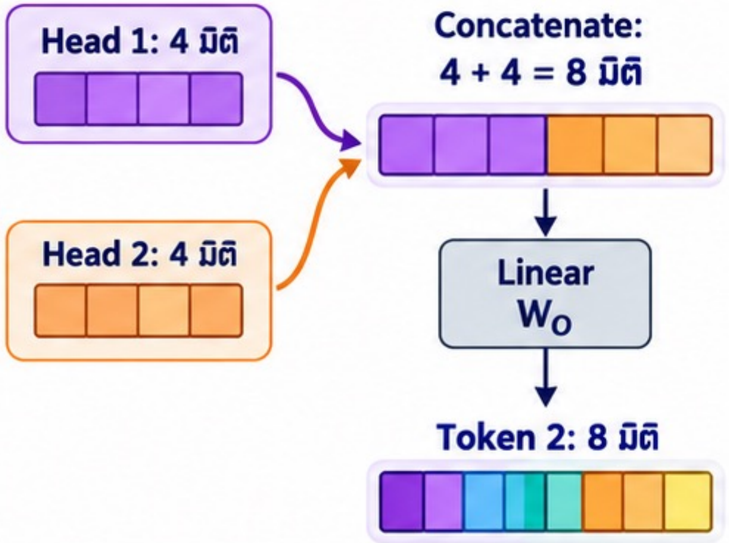
$$Z_h = A_h V_h$$

แต่ละ Head รวมข้อมูลตามน้ำหนัก Attention

ทำเช่นนี้กับทุก Token

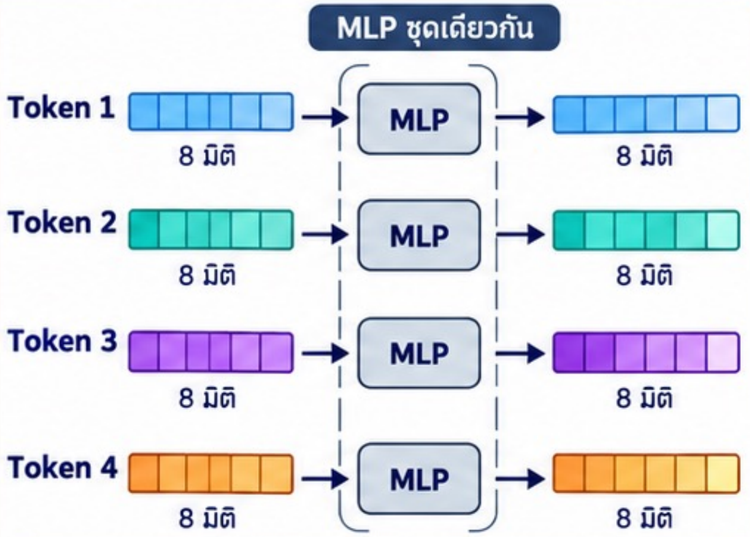
2. Concat + W_O: รวมผลจาก Heads

Token 2 เดียวกัน



Concat ต่อเวกเตอร์ • W_O ผสมคุณลักษณะจาก Heads

3. MLP: แปลงแต่ละ Token

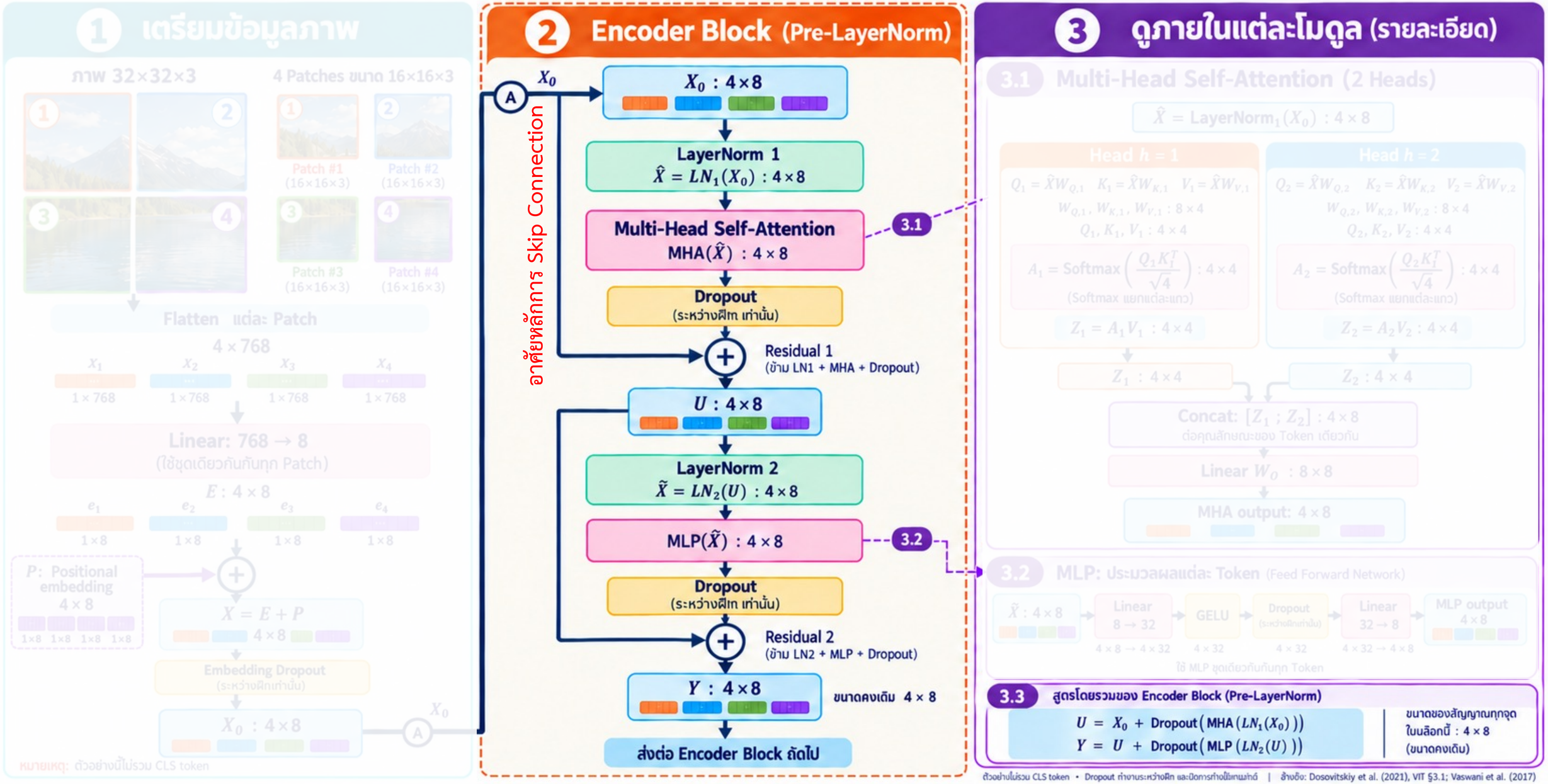


ภายใน MLP (เหมือนกันทุก Token)

Linear 8 → 32 → GELU → Linear 32 → 8

ผสมคุณลักษณะภายใน Token

ไม่รวมข้อมูลข้าม Tokens โดยตรง


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกับทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
X0
A
อาศัยหลักการ Skip Connection
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4×8
3.1
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4×8
3.2
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4×8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8×4
Q1, K1, V1 : 4×4
A1 = Softmax(Q1K1T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4×4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8×4
Q2, K2, V2 : 4×4
A2 = Softmax(Q2K2T/√4) : 4×4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4×4
Z1 : 4×4
Z2 : 4×4
Concat: [Z1 ; Z2] : 4×8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8×8
MHA output: 4×8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4×8
Linear 8 → 32
GELU
Dropout (ระหว่างฝึกเท่านั้น)
Linear 32 → 8
MLP output 4×8
4×8 → 4×32
4×32
4×32
4×32 → 4×8
ใช้ MLP ชุดเดียวกันกับทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานในการทำนาย | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

แนวคิดภายใน Encoder Block

Encoder Block ใช้กลไก Attention เพื่อแลกเปลี่ยนข้อมูลระหว่าง Tokens จากนั้น ใช้ MLP ปรับคุณลักษณะภายในแต่ละ Token ก่อนส่งผลลัพธ์ไปยัง Encoder Block ถัดไป

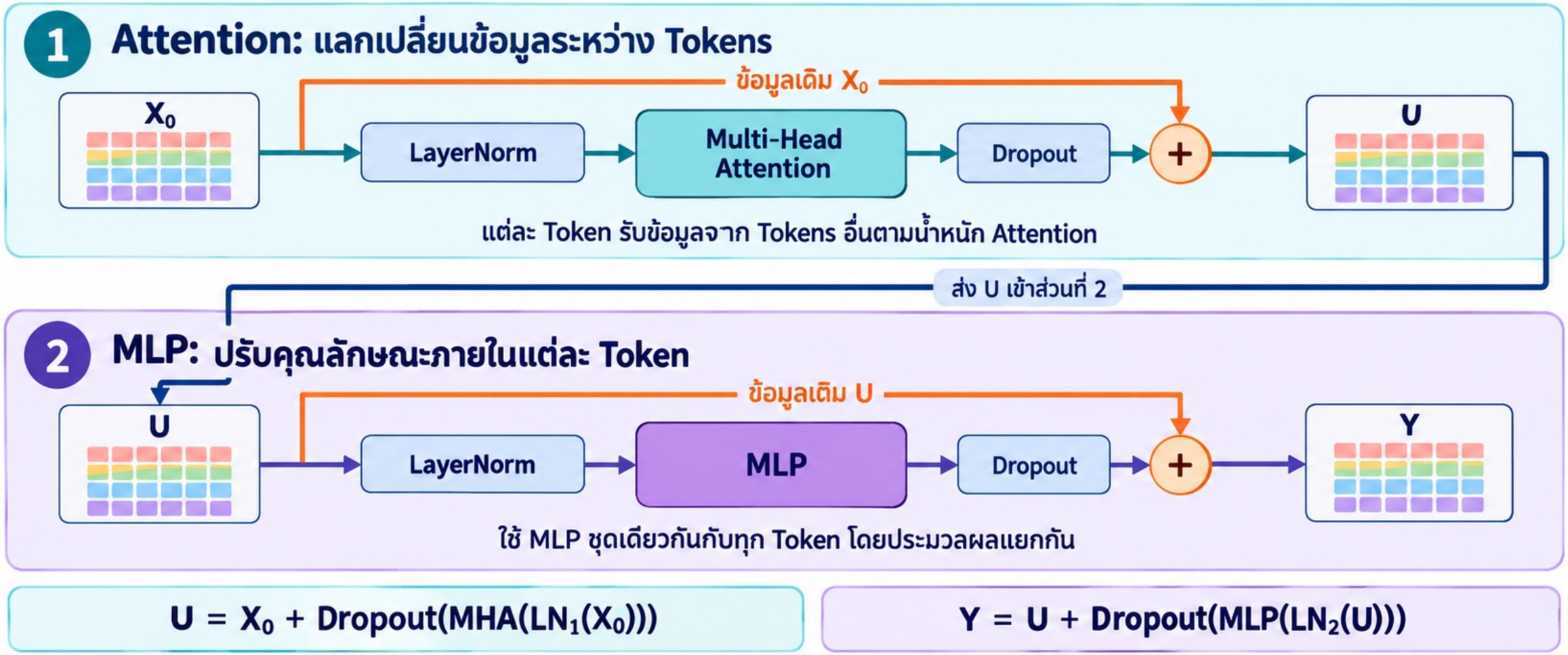
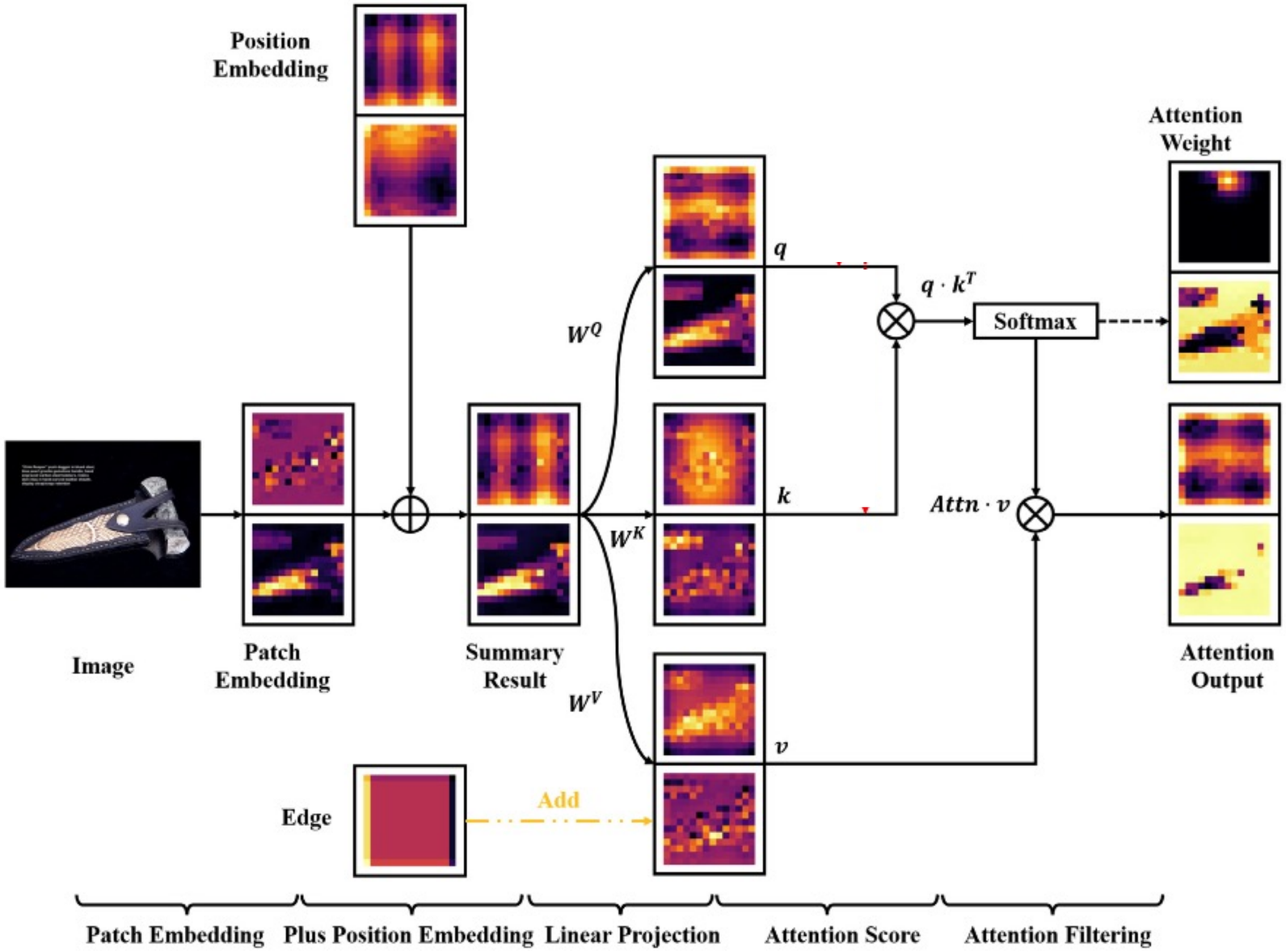
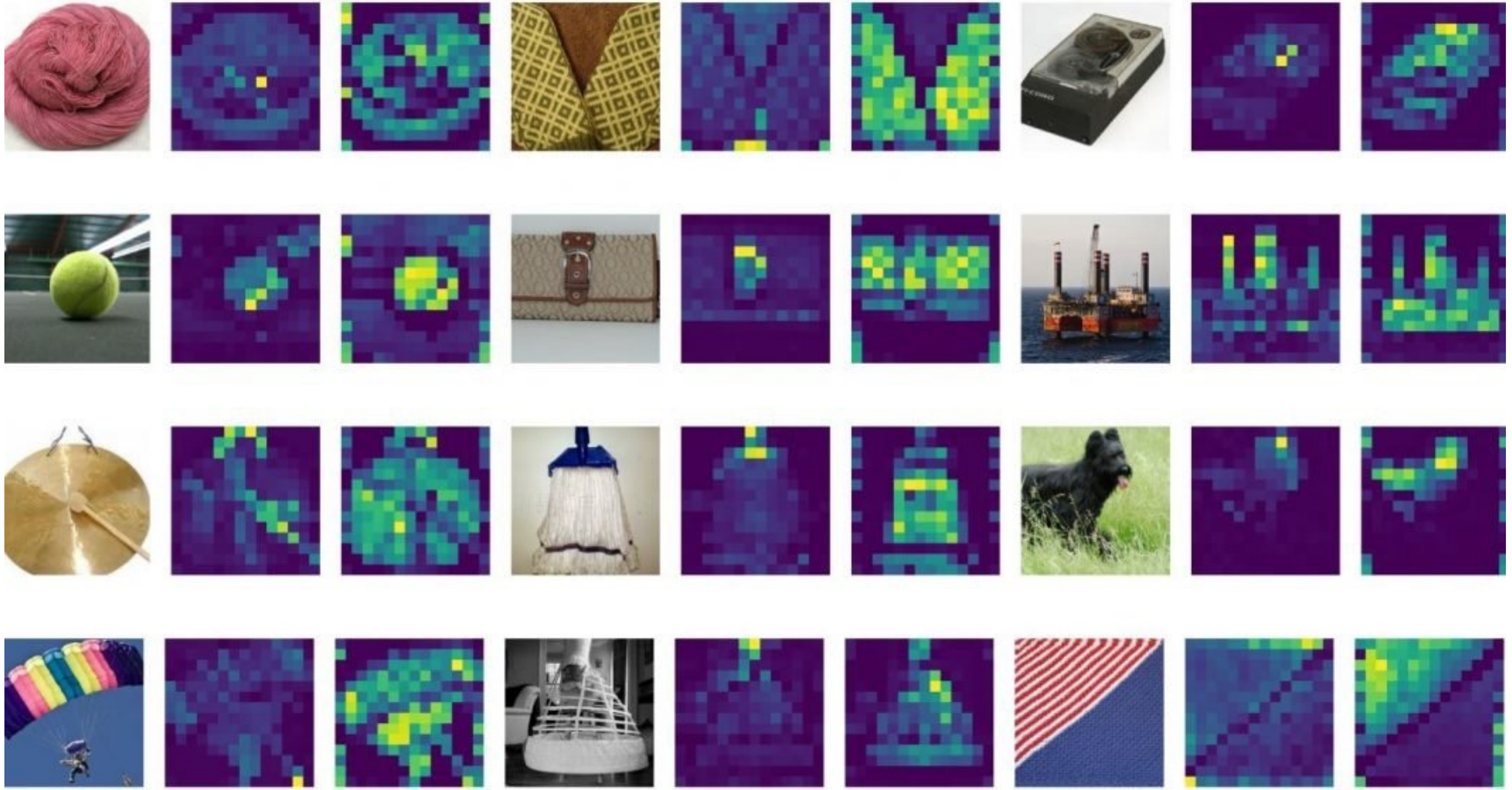
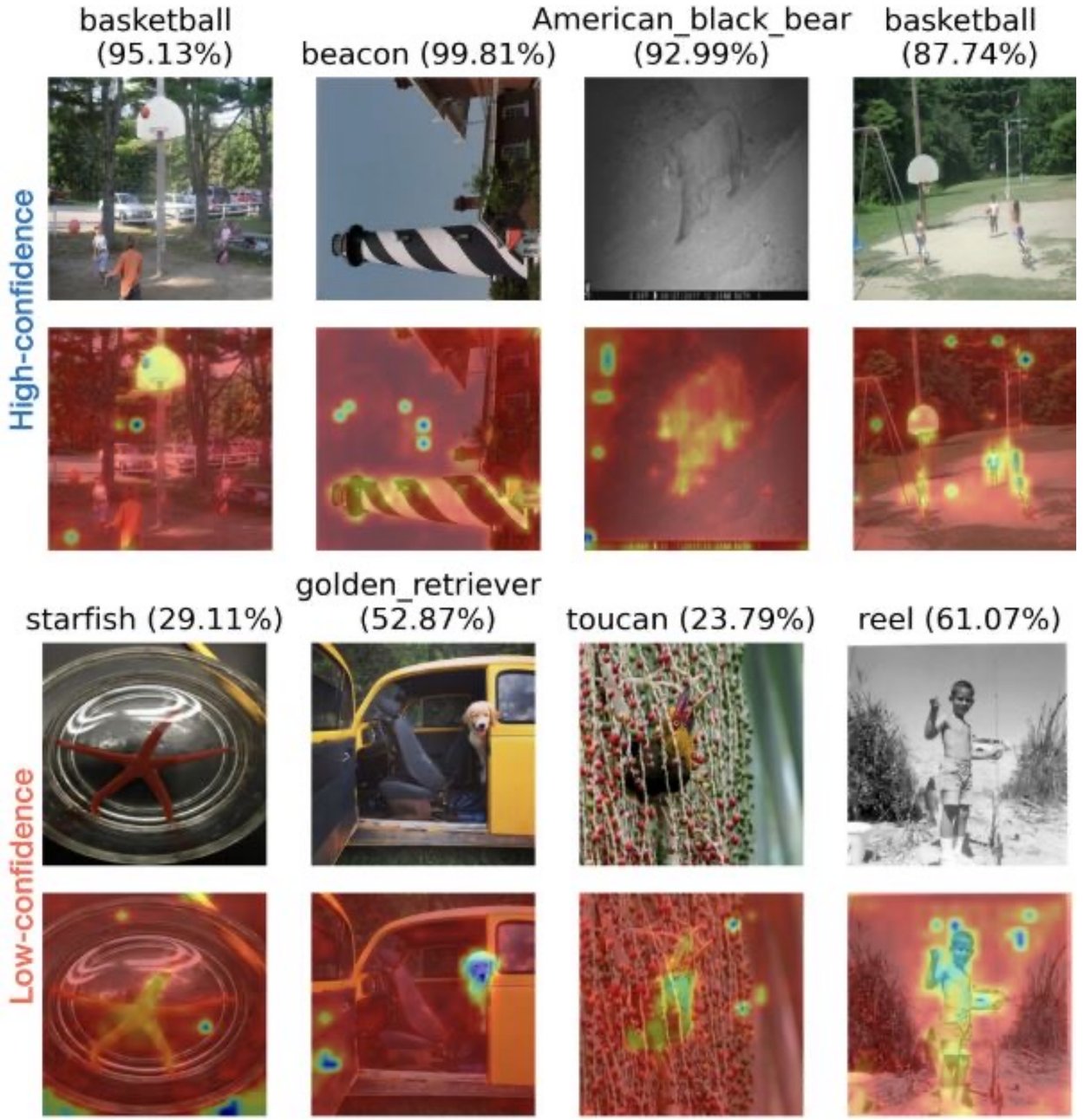


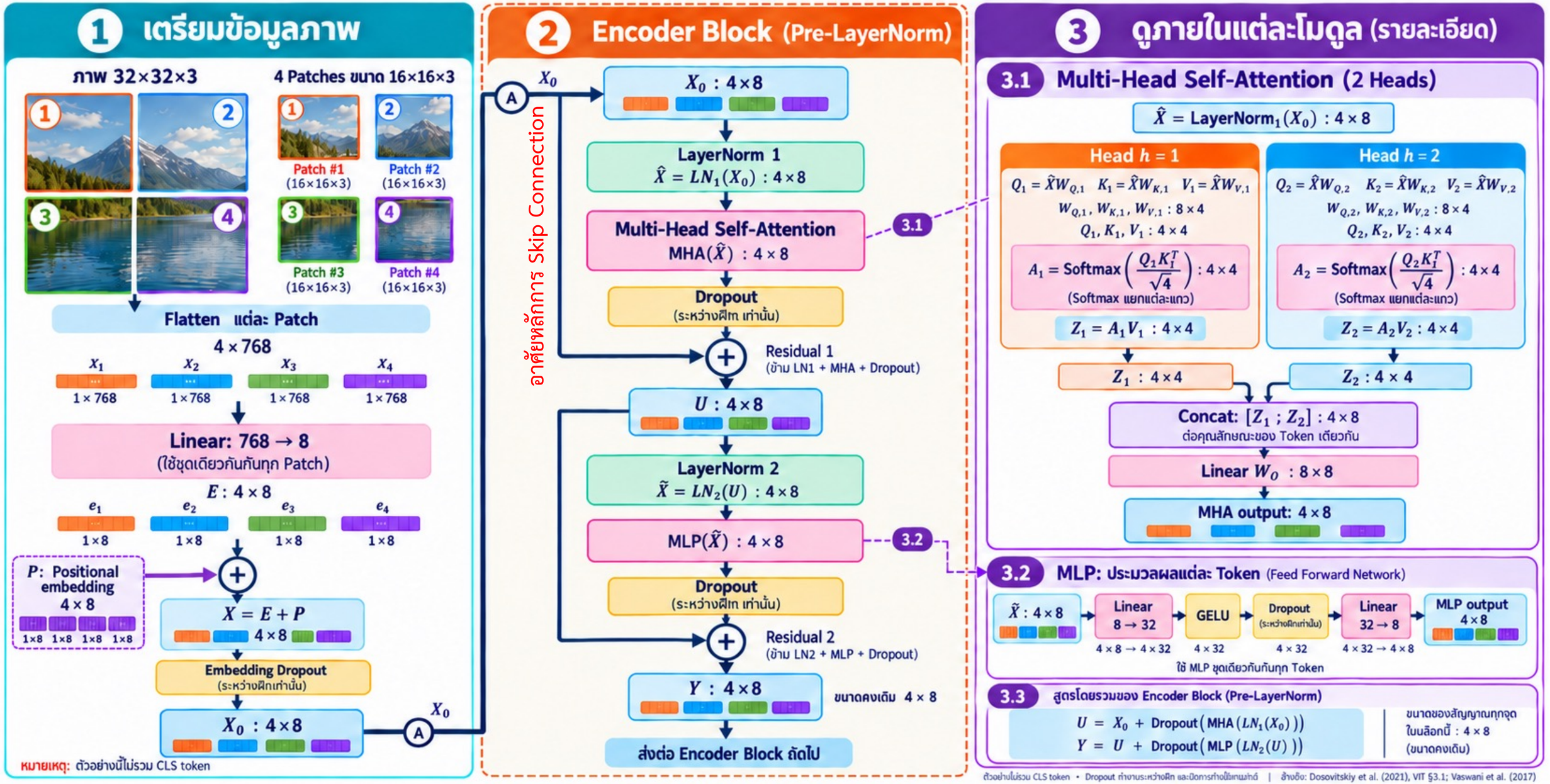
Image generated by OpenAI (2026).



อ้างอิง https://doi.org/10.1016/j.jvcir.2022.103664




basketball (95.13%)
beacon (99.81%)
American_black_bear (92.99%)
basketball (87.74%)
High-confidence
starfish (29.11%)
golden_retriever (52.87%)
toucan (23.79%)
reel (61.07%)
Low-confidence


1 เตรียมข้อมูลภาพ
ภาพ 32×32×3
4 Patches ขนาด 16×16×3
Patch #1 (16×16×3)
Patch #2 (16×16×3)
Patch #3 (16×16×3)
Patch #4 (16×16×3)
Flatten แต่ละ Patch
4 × 768
1 × 768
Linear: 768 → 8
(ใช้ชุดเดียวกันกันทุก Patch)
E : 4 × 8
1×8
P: Positional embedding
4 × 8
X = E + P
4 × 8
Embedding Dropout
(ระหว่างฝึกเท่านั้น)
X0 : 4×8
หมายเหตุ: ตัวอย่างนี้ไม่รวม CLS token
2 Encoder Block (Pre-LayerNorm)
อาศัยหลักการ Skip Connection
X0 : 4×8
LayerNorm 1
X̂ = LN1(X0) : 4×8
Multi-Head Self-Attention
MHA(X̂) : 4×8
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 1
(ข้าม LN1 + MHA + Dropout)
U : 4×8
LayerNorm 2
X̃ = LN2(U) : 4×8
MLP(X̃) : 4×8
Dropout
(ระหว่างฝึก เท่านั้น)
Residual 2
(ข้าม LN2 + MLP + Dropout)
Y : 4×8
ขนาดคงเดิม 4 × 8
ส่งต่อ Encoder Block ถัดไป
3 ดูภายในแต่ละโมดูล (รายละเอียด)
3.1 Multi-Head Self-Attention (2 Heads)
X̂ = LayerNorm1(X0) : 4 × 8
Head h = 1
Q1 = X̂WQ,1 K1 = X̂WK,1 V1 = X̂WV,1
WQ,1, WK,1, WV,1 : 8 × 4
Q1, K1, V1 : 4 × 4
A1 = Softmax(Q1K1T / √4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z1 = A1V1 : 4 × 4
Head h = 2
Q2 = X̂WQ,2 K2 = X̂WK,2 V2 = X̂WV,2
WQ,2, WK,2, WV,2 : 8 × 4
Q2, K2, V2 : 4 × 4
A2 = Softmax(Q2K2T / √4) : 4 × 4
(Softmax แยกแต่ละแถว)
Z2 = A2V2 : 4 × 4
Z1 : 4×4
Z2 : 4 × 4
Concat: [Z1 ; Z2] : 4 × 8
ต่อคุณลักษณะของ Token เดียวกัน
Linear WO : 8 × 8
MHA output: 4 × 8
3.2 MLP: ประมวลผลแต่ละ Token (Feed Forward Network)
X̃ : 4 × 8
Linear 8 → 32
4 × 8 → 4 × 32
GELU
4 × 32
Dropout (ระหว่างฝึกเท่านั้น)
4 × 32
Linear 32 → 8
4 × 32 → 4 × 8
MLP output 4 × 8
ใช้ MLP ชุดเดียวกันกันทุก Token
3.3 สูตรโดยรวมของ Encoder Block (Pre-LayerNorm)
U = X0 + Dropout(MHA(LN1(X0)))
Y = U + Dropout(MLP(LN2(U)))
ขนาดของสัญญาณทุกจุดในบล็อกนี้ : 4 × 8 (ขนาดคงเดิม)
ตัวอย่างไม่รวม CLS token • Dropout ทำงานระหว่างฝึก และปิดการทำงานเมื่อทำนาย | อ้างอิง: Dosovitskiy et al. (2021), ViT §3.1; Vaswani et al. (2017)

การจัดเรียง Encoder Block

การนำหลาย Encoder Block มาเรียงต่อกัน ช่วยให้โมเดลเรียนรู้และปรับความสัมพันธ์ระหว่าง Tokens ได้หลายขั้น โดยแต่ละ Block ใช้คุณลักษณะที่ผ่านการปรับจาก Block ก่อนหน้า เพื่อสร้างความเข้าใจบริบทและรูปแบบที่ซับซ้อนขึ้น อย่างไรก็ตาม จำนวน Block ที่มากขึ้นไม่ได้รับประกันว่าผลลัพธ์จะดีขึ้นเสมอไป และเพิ่มต้นทุนการคำนวณด้วย

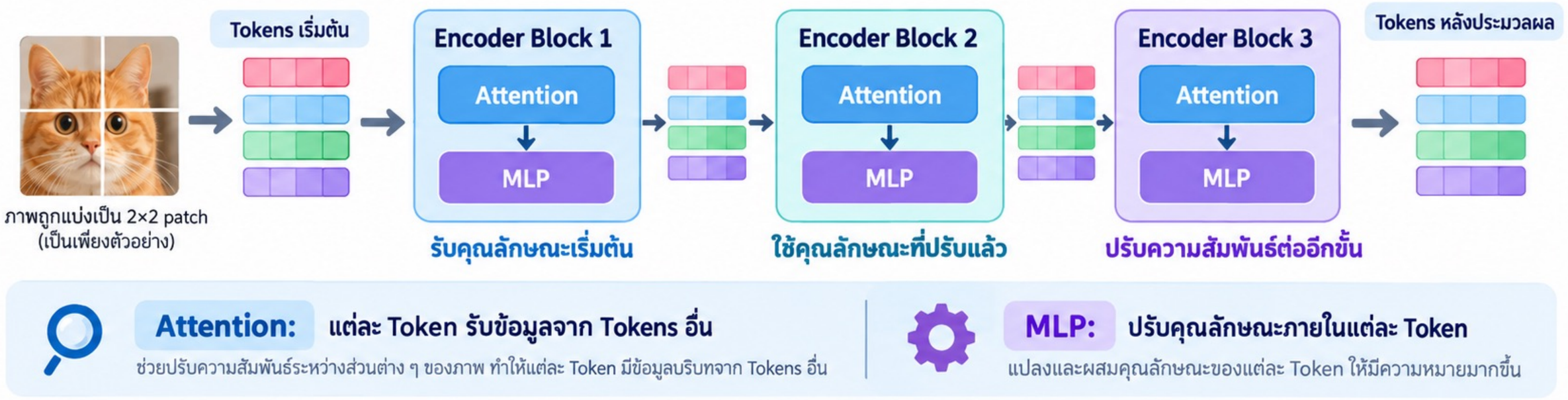


Image generated by OpenAI (2026).

การนำผลลัพธ์จาก Encoder Block ไปประยุกต์ใช้งาน

หลังผ่าน Encoder Block สุดท้าย เราจะได้ เวกเตอร์คุณลักษณะของแต่ละ Token ที่มีข้อมูลบริบทจากภาพแล้ว จากนั้นต้องสร้าง เวกเตอร์ตัวแทนภาพหนึ่งเวกเตอร์ แล้วส่งเข้า Classification Head (ส่วนใหญ่จะเป็น Linear จำนวน 1 เลเยอร์) เพื่อทำนายผลลัพธ์

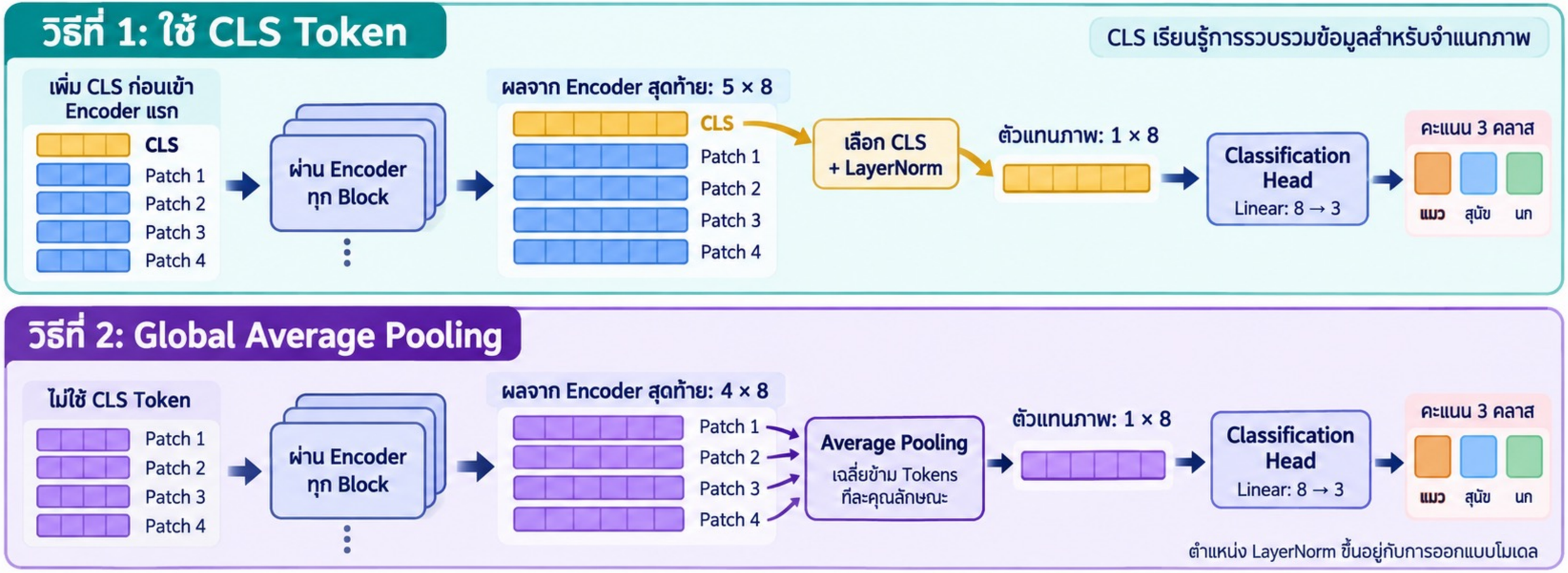
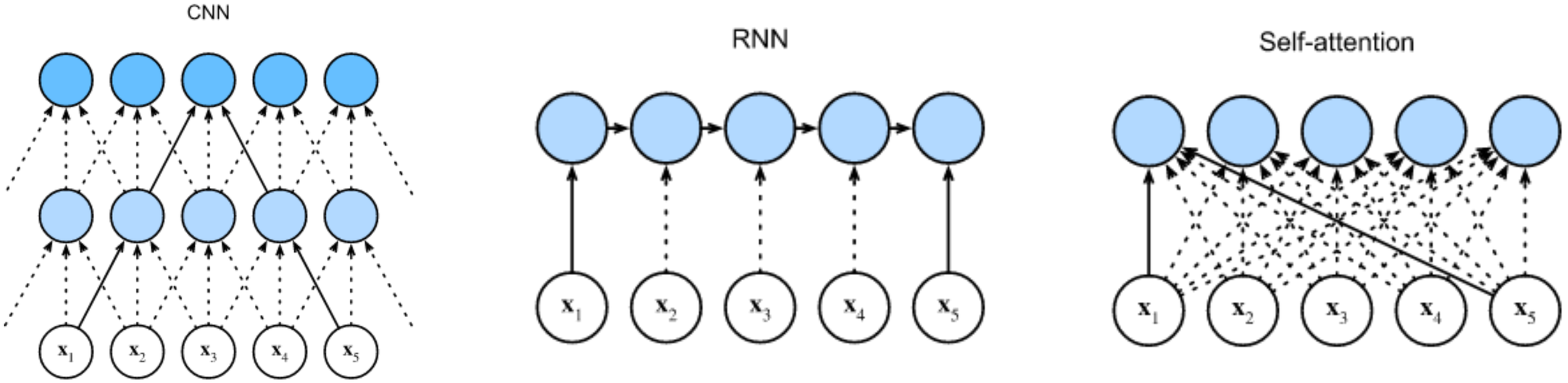


Image generated by OpenAI (2026).

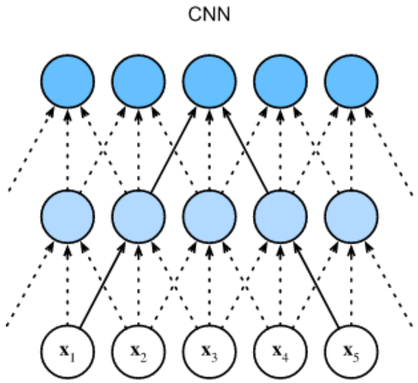
ความแตกต่างระหว่าง CNN, RNN, Self-attention

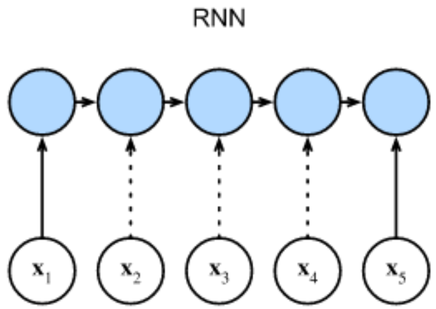


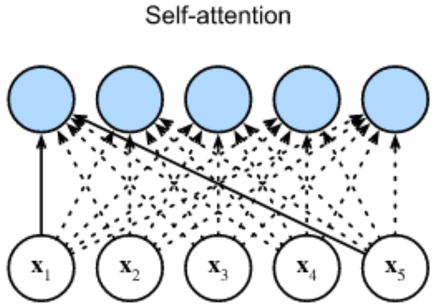
- **CNN** มีการประมวลเชิงลำดับชั้น (Hierarchical) และเป็นการประมวลผลเฉพาะท้องถิ่น ซึ่งรองรับการประมวลผลแบบขนาน (Parallel)
- **RNN** เป็นการประมวลผลแบบเป็นลำดับ (Sequence) ไม่รองรับการประมวลผลแบบขนาน (Parallel)
- **Self-attention** เป็นการประมวลเฉพาะโหนดที่มีความสัมพันธ์กันโดยตรง ซึ่งรองรับการประมวลผลแบบขนาน (Parallel)

อ้างอิง https://d2l.ai/chapter_attention-mechanisms-and-transformers/self-attention-and-positional-encoding.html

ความแตกต่างระหว่าง CNN, RNN, Self-attention







	CNN	RNN	Self-Attention
Complexity per Layer	$O(k \cdot n \cdot d^2)$	$O(n \cdot d^2)$	$O(n^2 d)$
Sequential Operations	$O(1)$	$O(n)$	$O(1)$
Maximum Path Length	$O(log_k(n))$	$O(n)$	$O(1)$

โดยที่ n คือ ความยาวของ Input Sequence, d คือ จำนวน Channel ของ Input และ Output, k คือ ขนาดของ kernel สำหรับการ convolution

อ้างอิง https://d2l.ai/chapter_attention-mechanisms-and-transformers/self-attention-and-positional-encoding.html

ความแตกต่างระหว่าง CNN และ Transformer

CNN แบบทั่วไปใช้ Kernel ที่เรียนรู้ไว้ในการรวมข้อมูลจากบริเวณใกล้เคียง ส่วน Transformer ใช้ Self-attention คำนวณน้ำหนักการรวมข้อมูลระหว่าง Patch ตามเนื้อหาของภาพ จึงรองรับความสัมพันธ์ที่เปลี่ยนไปตามข้อมูลเข้าได้ โดยพารามิเตอร์ของทั้งสองโมเดลยังคงเดิมระหว่างการใช้งานหลังฝึกสอน

สิ่งที่เปรียบเทียบ	CNN	Transformer
พารามิเตอร์หลังฝึกสอน	ค่าใน Kernel คงเดิม	Wq, Wk, Wv และพารามิเตอร์อื่นคงเดิม
สิ่งที่เปลี่ยนตามภาพ *input*	Feature map และผลทำนาย	Q, K, V, Attention map และผลทำนาย
การรวมข้อมูลระหว่างตำแหน่ง	ใช้ Kernel ที่เรียนรู้ไว้กับบริเวณที่กำหนด	คำนวณน้ำหนักระหว่าง Token จากข้อมูลของ Token
บริเวณที่เชื่อมโยงได้ในหนึ่งชั้น	ขึ้นอยู่กับขนาด Kernel และ Dilation	Global self-attention เชื่อมโยงทุก Patch ได้

Image generated by OpenAI (2026).

ความแตกต่างระหว่าง CNN และ Transformer

CNN: Convolution ปกติ



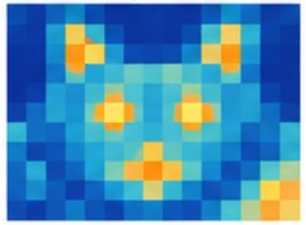
ภาพแมวด้านหน้า

Kernel เดิม
(3 × 3)



ใช้ค่าน้ำหนักเดิม
ใน Kernel

Feature map เปลี่ยนตามภาพ



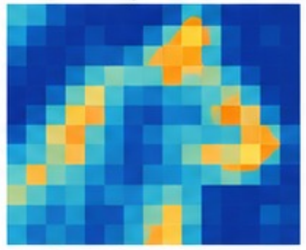


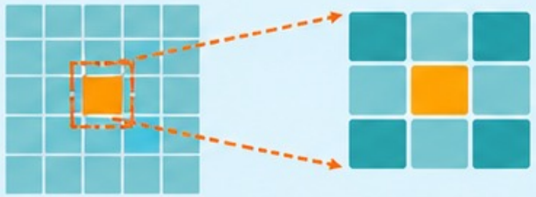
ภาพแมวด้านข้าง

Kernel เดิม
(3 × 3)



Feature map เปลี่ยนตามภาพ





รวมข้อมูลจากบริเวณที่ Kernel ครอบคลุม

ขอบเขตขึ้นอยู่กับ Kernel และ Dilation

Transformer: Self-attention



ภาพแมวด้านหน้า

W_Q, W_K, W_V
คงเดิม

Q K V

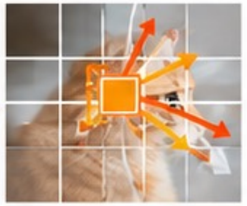




ภาพแมวด้านข้าง

W_Q, W_K, W_V
คงเดิม

Q K V



Q, K, V และน้ำหนัก Attention
เปลี่ยนตามภาพ

คำนวณน้ำหนักการรวมข้อมูล
จาก Token แต่ละคู่

Global: เชื่อมโยงทุก Patch ได้



Token ที่เลือก
สามารถเชื่อมโยง
กับทุก Patch
ในภาพได้

Window: ภายในหน้าต่าง

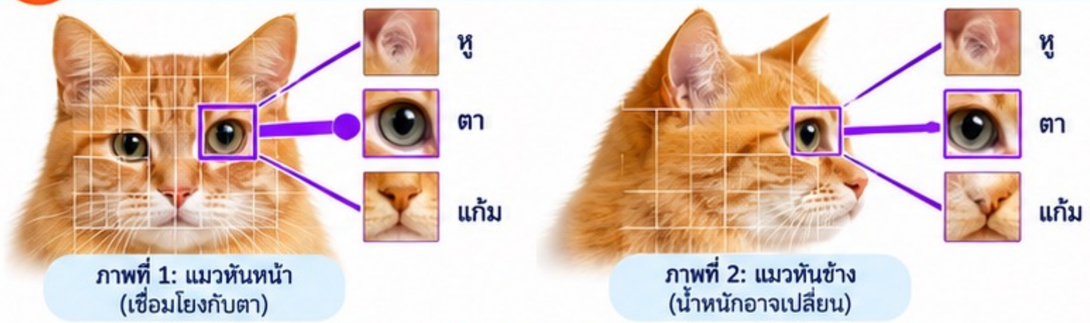


เชื่อมโยงเฉพาะ
Patch ในหน้าต่าง
ที่กำหนด

Image generated by OpenAI (2026).

ความแตกต่างระหว่าง CNN และ Transformer

1 ปรับน้ำหนักตามเนื้อหา

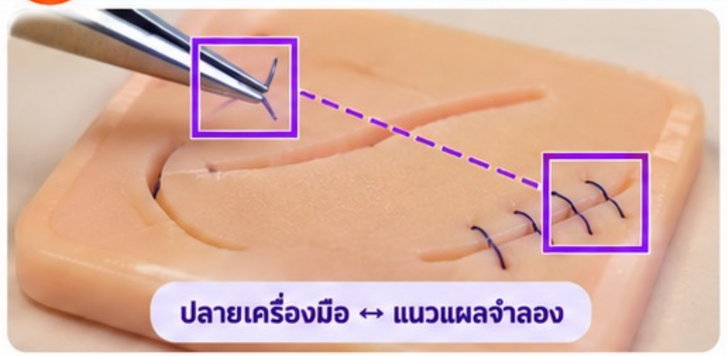


ภาพเปลี่ยน → น้ำหนัก Attention อาจเปลี่ยน

CNN: ใช้ค่าน้ำหนักเดิมใน Kernel

Attention: คำนวณน้ำหนักระหว่าง Patch ตามข้อมูล

2 เชื่อมโยงบริเวณที่อยู่ไกล







CNN: Kernel เล็กต้องอาศัยหลายชั้นเพื่อเห็นกว้างขึ้น

Global attention: เชื่อม Patch ที่ไกลกันได้ในชั้นเดียว

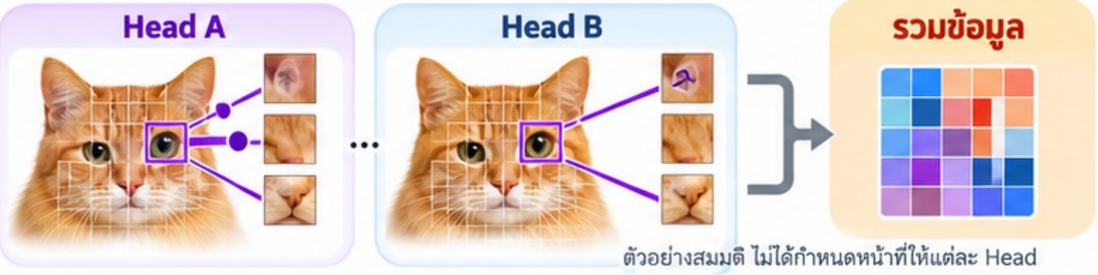
3 ใช้บริบทช่วยแยกสิ่งที่คล้ายกัน



CNN: บริเวณเล็กอาจมีข้อมูลไม่พอ

Attention: ใช้ข้อมลจากบริเวณอื่นช่วยตีความ

4 รวมข้อมูลหลายความสัมพันธ์



ตัวอย่างสมมติ ไม่ได้กำหนดหน้าที่ให้แต่ละ Head

CNN: มีหลาย Filter แต่เชื่อมตำแหน่งตาม Kernel

Multi-head: เรียนรู้รูปแบบการรวมข้อมูลได้หลายแบบ

เป็นข้อได้เปรียบเชิงกลไก ผลจริงขึ้นอยู่กับข้อมูล สถาปัตยกรรม และการฝึก

Image generated by OpenAI (2026).